AASTU

Addis Ababa Science and Technology University
College of Engineering

Department of Software Engineering

AI-POWERED LUNG DISEASE DETECTION AND CLINICAL DECISION SUPPORT SYSTEM

AASTU

Addis Ababa Science and Technology University

College of Engineering

Department of Software Engineering

AI-POWERED LUNG DISEASE DETECTION AND CLINICAL DECISION SUPPORT SYSTEM

Group Members:

1	**Mahlet Assefa**	**ETS1000/14**
2	**Meseret Bolled**	**ETS1052/14**
3	**Michael Addis**	**ETS1060/14**
4	**Michael Workineh**	**ETS1058/14**
5	**Michael Getu**	**ETS1061/14**

Adviser: Inst. Getnet **Signiture:**

Date: Jan 20 2026

Acknowledegement

We would like to thank our professors and adviser for their important advice, helpful criticism, and unwavering support during this endeavor. Their expertise and encouragement greatly contributed to the successful completion of this work.

We also acknowledge Beijing hospital and Black lion hospital that provided datasets for our work. We would especially want to thank the authors and researchers whose open-source implementations and publicly accessible datasets made this project possible.

Lastly, we would like to thank our friends and family for their encouragement and moral support throughout the project's development.

Table of Content

List of Tables

List of images

List of Abbreviations

- Advanced Encryption Standard (AES)
- Application programming interface (API)
- Artificial Intelligence (AI)
- Atomicity, Consistency, Isolation, and Durability (ACID)
- Computed tomography (CT)
- Computer-aided diagnosis (CAD)
- Computer-aided diagnosis (CAD)
- Contrast Limited Adaptive Histogram Equalization (CLAHE)
- Convolutional Neutral Network (CNN)
- Depthwise Convolution (DwConv)
- Graphics Processing Unit (GPU)
- Hospital information systems (HIS).
- Institute of Electrical and Electronics Engineers (IEEE)
- JavaScript Object Notation (JSON)
- JSON Web Token (JWT)
- Large Cell Carcino (LCC)
- Mean Average Precision (mAP)
- Residual neural network (ResNet)
- Support vector machine (SVM)
- Transport Layer Security (TLS)
- Tuberculosis (TB)
- Unified Modeling Language (UML)
- User Interface (UI)
- Visual Geometry Group (VGG)

- Web Content Accessibility Guidelines (WCAG)
- World Health Organization (WHO)
- You Only Look Once (YOLO)

Chapter One: Introduction

Lung diseases, including tuberculosis (TB), pneumonia, and lung cancer, represent a significant public health challenge globally, but their impact is particularly acute in low- and middle-income countries like Ethiopia. According to the World Health Organization (WHO), Ethiopia ranks among the high-burden countries for TB, with an estimated incidence of 132 cases per 100,000 population in 2023, contributing to over 20,000 deaths annually[8]. Pneumonia, often linked to underlying conditions such as HIV/AIDS and malnutrition, remains a leading cause of mortality among children under five, accounting for approximately 15% of under-five deaths in the country. Lung cancer, though less prevalent due to lower tobacco use rates compared to Western nations, is on the rise with urbanization and increasing exposure to environmental risk factors like biomass fuel smoke, with incidence rates estimated at 4-6 cases per 100,000 population[9]. These diseases collectively strain Ethiopia's healthcare system, which is characterized by limited diagnostic infrastructure and a severe shortage of specialized medical personnel.

Current diagnostic practices in Ethiopia rely heavily on chest X-rays and, to a lesser extent, computed tomography (CT) scans for detecting pulmonary abnormalities. However, the existing system faces substantial shortcomings: manual interpretation by radiologists is time-consuming, prone to human error (with error rates of 20-30% in routine practice), and inaccessible in rural and underserved areas where over 80% of the population resides. Ethiopia has fewer than 200 radiologists for a population exceeding 120 million, leading to diagnostic delays that exacerbate disease progression and mortality[10]. Attempted solutions, such as telemedicine initiatives and basic computer-aided diagnosis (CAD) tools piloted by organizations like the WHO and Partners In Health, have shown promise but are limited by proprietary software, high costs, lack of integration with local hospital systems, and insufficient accuracy for multi-disease detection. These gaps highlight the need for an indigenous, AI-driven solution tailored to Ethiopia's resource-constrained environment.

This project proposes the development of an AI-powered infrastructure utilizing the You Only Look Once (YOLO) object detection model to enable hospitals across Ethiopia to upload chest X-ray and CT images for automated detection of lung cancer, pneumonia, and tuberculosis. By leveraging deep learning for real-time analysis, the

system aims to bridge the diagnostic divide, enhancing early detection and clinical decision-making in both urban and rural settings.

1.1. Statement of the Problem

The accurate and timely diagnosis of lung diseases such as tuberculosis, pneumonia, and lung cancer in Ethiopia is hindered by several interconnected challenges that compromise public health outcomes. Primarily, there is a profound shortage of radiologists and diagnostic imaging experts, with ratios as low as one radiologist per 350,000 people, resulting in overwhelming backlogs at major hospitals and virtually no access in remote regions [10]. This scarcity leads to delayed diagnoses, where patients with treatable conditions like TB often present at advanced stages, increasing transmission rates and healthcare costs.

Furthermore, manual interpretation of chest X-rays and CT scans is susceptible to high error rates (20-30%) due to factors such as image quality variations from outdated equipment, clinician fatigue in high-volume settings, and the subtlety of early-stage pathologies. [11] In Ethiopia, where TB and pneumonia are endemic, these errors contribute to misdiagnoses, unnecessary treatments, and preventable deaths—exemplified by the country's annual TB mortality rate exceeding 20,000 despite available treatments [12].

Existing AI-based diagnostic tools, while emerging globally, are largely inaccessible in Ethiopia due to their proprietary nature, high licensing fees, and incompatibility with local infrastructure. These systems often lack explainability, failing to provide clinicians with transparent reasoning or integration of patient clinical history, which erodes trust and limits adoption. Additionally, there is no standardized platform for hospitals to securely upload and analyze images, leading to fragmented data silos and missed opportunities for national-level disease surveillance.

Without an affordable, scalable AI solution tailored to Ethiopian contexts, for instance variable image quality from mobile X-ray unit, these problems will persist, perpetuating health inequities and straining the national healthcare budget. This study addresses this critical gap by developing a YOLO-based infrastructure for multi-disease detection, aiming to improve diagnostic efficiency, accuracy, and accessibility nationwide.

1.2. Objectives

The objectives of this study are designed to guide the development of a robust AI-powered system for lung disease detection, ensuring alignment with Ethiopia's healthcare needs.

1.2.1. General Objective

To develop an AI-powered infrastructure using the YOLO model for hospitals in Ethiopia to upload chest CT images and chest X-rays for the detection of lung cancer, pneumonia, and tuberculosis, thereby enhancing diagnostic accuracy, speed, and accessibility in resource-limited settings.

1.2.2. Specific Objectives

- Conduct requirement gathering and analysis by engaging stakeholders from Ethiopian hospitals and public datasets to identify user needs, data privacy requirements, and integration points with existing hospital information systems (HIS).
- Design a hybrid YOLO-based architecture that combines object detection for lesion localization with classification mechanisms, incorporating data preprocessing pipelines for handling low-quality images common in Ethiopian facilities.
- Implement the system as a web-based platform allowing secure image uploads, real-time AI analysis, and generation of explainable outputs such as bounding boxes and clinical recommendations.
- Test and validate the model using integrated datasets from global sources (e.g., ChestX-ray14, TBX11K) augmented with local Ethiopian data, achieving at least 90% mAP50 core, while conducting user acceptance testing in pilot hospitals.
- Deploy the infrastructure in a scalable manner, including cloud-based hosting for nationwide access, training modules for clinicians, and mechanisms for ongoing model updates based on new data.

1.3. Scope and Limitation

The scope of this study encompasses the development and pilot deployment of a YOLO-based AI infrastructure focused on detecting lung cancer, pneumonia, and tuberculosis from chest X-rays and CT images uploaded by Ethiopian hospitals. It includes data collection from public datasets and local sources, model training for multi-disease classification, and creation of a user-friendly web platform with secure authentication. The system will support integration with mobile devices for rural clinics and provide differential diagnoses based on AI predictions and basic patient metadata.

Limitations include the exclusion of real-time video analysis or other imaging modalities (e.g., MRI), as the focus is on static X-rays and CT scans. Due to resource constraints, full integration with all Ethiopian hospital electronic health records (EHR) systems will not be implemented; instead, a basic API interface will be provided. The model may underperform on extremely rare variants of diseases not well-represented in training data, and ethical approvals for using local patient data may delay full validation. Hardware dependencies, such as reliance on cloud computing, could pose challenges in areas with unreliable internet, though offline modes were scoped but not fully realized due to computational demands.

1.4. Methodology

This study employs a mixed-methods approach combining quantitative model development with qualitative stakeholder engagement. The research design follows an agile software development lifecycle, iterating through requirements, design, implementation, testing, and deployment phases.

Data collection will involve sourcing public datasets (e.g., ChestX-ray14 with over 100,000 images, TBX11K) and collaborating with Ethiopian hospitals for anonymized local data, ensuring compliance with data protection regulations. Preprocessing techniques, including augmentation (rotation, flipping) and enhancement (CLAHE for contrast), will address image variability.

For model development, YOLOX will be customized for object detection and localization, integrated with CNN classifiers (e.g., EfficientNet) for disease categorization. The technology stack includes Python with libraries such as PyTorch, OpenCV, and FastAPI for the web backend; frontend will use React.js for the upload interface. Cloudco with min.io, with Docker for containerization.

Analysis strategies will use metrics like mean Average Precision (mAP), sensitivity, and specificity. Testing will follow IEEE standards, including unit, integration, and system testing, with user trials in clinical settings. Ethical considerations, per Ethiopian Health Research Ethics guidelines, will prioritize patient privacy and informed consent.

1.5. Plan of Activities

The project will span 4 months, from December 2025 to March 2026, divided into quarters with key milestones.

- Quarter 1 (December 2025): Requirement gathering, literature review completion, and dataset collection. Milestone: Approved project proposal and initial data pipeline.
- Quarter 2 (January 2026): Model design, preprocessing implementation, and initial training. Milestone: Prototype YOLO model with 80% accuracy on test data.
- Quarter 3 (February 2026): System implementation, web platform development, and integration of explainable features. Milestone: Functional beta version ready for internal testing.
- Quarter 4 (March 2026): Validation, pilot deployment in hospitals, and final refinements. Milestone: Deployed system with evaluation report.

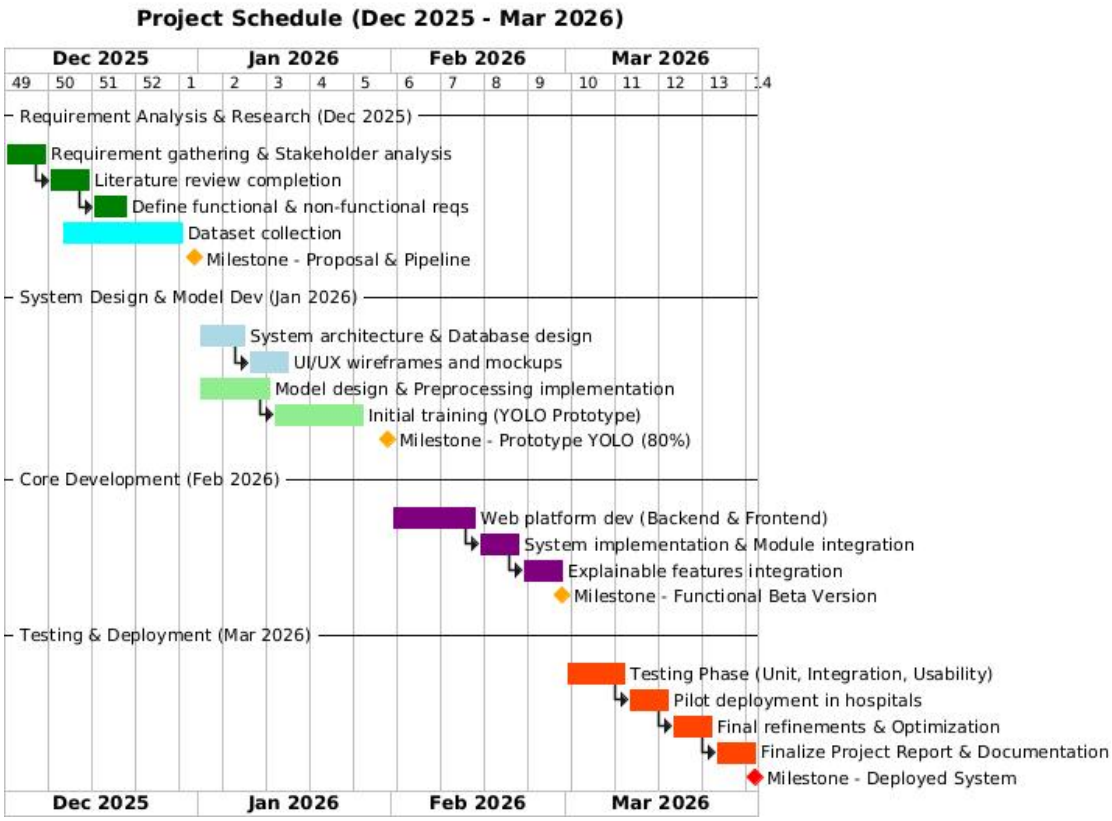

Image 1 Gantt chart

1.6. Budget Required

This project is designed to be implemented at zero monetary cost by fully utilizing free and open-source tools, university resources, and publicly available datasets. No purchase of hardware, licensed software, or paid cloud services is required.

Category	Estimated Cost(ETB)	Used
Hardware/ Computing Power	0	• Google Colab Pro / free tier (provides free NVIDIA T4, P100, or A100 GPUs with 12–30 GB RAM) • Kaggle Notebooks (free 40 hours/week of GPU/TPU)
Software & Frameworks	0	• Python, PyTorch, OpenCV, FastAPI, Streamlit, React.js – all completely free and open-source • VS Code, Jupyter, Git/GitHub – free
Dataset	0	• Public datasets: ChestX-ray14, CheXpert, RSNA Pneumonia, TBX11K, COVID-19 Radiography Database, VinBigData, MIMIC-CXR (all free for research)

		• Local anonymized data collected with ethical clearance from partner hospitals (no payment required)
Cloud & Hosting	0	• Backend & model inference: free tier of Render.com, Railway.app, or Fly.io • Alternative: free Oracle Cloud Always-Free tier (2 VMs + 200 GB storage) • Frontend hosting: Vercel or Netlify free plan • Model storage: Hugging Face Hub (free)
Data Annotation	0	• Use free tools: LabelImg, CVAT (self-hosted), Makesense.ai, Roboflow (free tier)
Development & Collaboration Tools	0	• GitHub (unlimited free private repositories for students) • Notion / Google Docs for documentation • Mattermost/ Telegram for team communication
Travel	50,000 Birr	• Hospital visits within Addis Ababa covered by students' personal transport • Regional visits replaced with digital meetings or data shared via encrypted Google Drive
Laber Costs	90,000 Birr	• Team Members: Assuming each team member works 10 hours per week on the project for 18 weeks. • Hourly Rate: 100 Birr/hour (average part-time rate for students).
Total Cost	140,000 Birr	

Table 1 Budget plan

1.7. Significance of the Study

This study holds substantial importance for advancing healthcare in Ethiopia and beyond. Scientifically, it contributes to the field of medical AI by demonstrating the efficacy of YOLO-based models in multi-disease detection. In healthcare, it addresses diagnostic shortages, reducing mortality from TB, pneumonia, and lung cancer by enabling early intervention which can potentially saving thousands of lives annually and alleviating hospital burdens. Practically, the open-source infrastructure will empower small clinics with affordable tools, fostering digital health equity. Broader implications include supporting Ethiopia's national health strategy for universal coverage, enhancing disease surveillance for outbreaks, and serving as a model for other African nations facing similar challenges.

1.8. Outline of the Study

This study is structured across multiple chapters to provide a comprehensive roadmap for the reader. Chapter One introduces the project, outlining the background, problem statement, objectives, scope, limitations, methodology, activities plan, budget, and significance. Chapter Two presents a detailed literature review on YOLO applications in pulmonary disease detection, highlighting gaps and lessons learned. Chapter Three describes problem analysis and modeling, which includes functional and non-functional requirements. Chapter Four describes the system analysis and design, including requirements specification and architectural diagrams. Chapter Five covers implementation details, such as code development and integration. Chapter Six discusses testing, validation, and results analysis. Chapter Seven offers conclusions, recommendations, and future work.

In recap, this introduction underscores the urgent need for AI-driven solutions to Ethiopia's lung disease diagnostic challenges, emphasizing the project's role in leveraging YOLO for accessible, accurate detection. It sets the foundation for subsequent chapters, which delve deeper into the technical and practical aspects to realize this innovative infrastructure.

Chapter Two: Literature Review

The literature review aims to serve as academic and empirical base for the project. A comprehensive overview of relevant research in the usage of the model You Only Look Once (YOLO) model in disease diagnosis using medical pulmonary medical imaging. It will analyze the strength, limitations, and trends detected in the literature we have chose to review.

The review highlights the extent of research advancement and academic landscape, clarifies the issues that need to be addressed. It attempts to articulate how the current research indicate the viability and potential of computer vision, YOLO, in pulmonary medication. Given that timely and accurate diagnosis of serious diseases like lung cancer, pneumonia, and tuberculosis is critical for effective treatment, this review tries to see the best possible approach for utilizing YOLO for pulmonary disease diagnosis.

2.1. Study Related Works

Early approaches for computer-aided diagnosis (CAD) devices often relayed on shallow classifiers and manually defined features. This has limited the capacity of generalization across different imaging conditions.[1] Later on, Convolutional Neutral Networks (CNNs) marked significant avancement. It showed notable success in automatically learning complex visual representations from unprocessed image data. (Manual feature extraction) [1]

Hybrid deep learning models that combine different architectures (VGG16 and VGG19) or integrate deep features with traditional machine learning classifiers (like SVMs) have been explored in attempt to enhance performance, although significant computation expense is needed. [1] Using ResNet50 and VGG19 for transfer learning has also become a commonly used strategy. This has been instrumental while working with limited medical image datasets. [1]

Due to YOLO's (You Only Look Once) highly notable real-time object detection ability and high precision, it has become a powerful approach for various medical detection tasks. Including these papers there have been many papers using the model for tuberculosis, pneumonia, and cancer diagnosis. [1] [2] [3]

2.2. Identifying Milestones of the Related Literatures and Finding the Gaps

A key mile stone is having an efficient and effective detection systems, that are based on the YOLO architecture, that is capable in balancing high accuracy with the speed and localization needed for fast diagnostics.

Ahmed et al. (2025) proposed a framework centered on the YOLOv11 architecture, which is intentionally designed for real-time pneumonia detection from chest X-ray images. It was capable to score a macro-average accuracy of 98.50% on the COVID-19 Radiography dataset [1] The central technological advancement is the integration of Grad-CAM for visual interpretability, this has made the model more trustworthy tool by providing clinicians visual justifications (heatmaps) for its predictions. [1] Additionally, a robust preprocessing pipeline, including Contrast Limited Adaptive Histogram Equalization (CLAHE) which is used for contrast enhancement and creating a focused lung segmentation. [1]

The paper has a gap in acknowledging the inherent complexity of medical image applications. The dataset used in the study covered limited range of patient demographics and imaging variations. [1]

Model	Accuracy(%)	Precision(%)	Recall(%)	F1-Score(%)
DenseNet201+SVM	96.29%	96.42%	96.42%	94.53%
VGG19	93.38%	94.12%	96%	95.05%
CNN	97.76%	97.78%	97.76%	97.76%
YOLOv11	98.50%	98.60%	97.45%	97.99%

Table 2 Performance evaluation against some related work using COVID-19 Radiography dataset [1]

A Fast-YOLO network model was proposed by Zhao et al. (2025) to optimize x-ray image detection by emphasizing computational efficiency. The model was capable have a detection speed 88 Frame Per Second while maintaining high accuracy.[2] A specific architectural innovation was the replacement of the compute-intensive C3k2 module in YOLOv11 with the lightweight FASPA Fast Pyramid Attention Mechanism. [2]

The major limitation was the lack of deep discussion or implementation regarding model interpretability. The reliability of the diagnostic networks in practise may face issues regarding generalization capability when the deployment environment's equipment or image quality is not synchronized with the simulation training set. [2]

Algorithms	Parameters	FPS	Prescision	Recall	mAP@0.5	mAP@0.5:0.95
YOLOv5n	1,789,624	93	94.1%	93.1%	97.0%	80.3%
YOLOv7-Tiny	6,062,584	35	95.3%	94.8%	97.8%	95.4%
YOLOv11	2,591,400	40	94.1%	95.2%	97.8%	97.8%
FAST-YOLO	2,279,193	88	93.4%	94.7%	97.7%	97.5%

Table 3 Performance comparison for the different algorithms [2]

A YOLOv7-based model was constructed by Bista et al. (2023) to construct a Computer-Aided Diagnosis (CAD) system specifically for Tuberculosis. A critical achievement was overcoming the severe data imbalance present in TBX11k dataset by applying both focal loss and minority class image augmentation. (rotation, scaling, and horizontal flipping) [4] The combination of the two approaches has resulted a promising mean average precision. The model was able to demonstrate robust generalization better than the previous model. [4]

Despite the performance improvements, the study also indicates more extensive and diverse dataset was needed to enhance its generalization capability. [4] The other persistant problem was learning curve, which showed a slight overfitting in boundary box prediction towards the end of the end of training. This indicates limitation in localizing lesions that could be mitigated more diverse data. [4]

Zeng et al. (2025) introduced YOLO-ED model, improvement over YOLOv8, tailored for fast and precise lung cancer detection in CT images. [5] Dual optimization is involved, where Efficient Modulation is replacing conventional C2F module to substantially reduce the computational load (by 2.4 GFLOPs compated to baseline YOLOv8) and substituting the standard upsampling with the DySample module to mitigate sampling blur and enhance detection (reaching 90.7% precision on the LUNG-PET-CT-DX dataset). [5] Lightweight design makes the model highly suitable for deployment in resource-constrained environments. [5]

The model experienced lower precision when detecting Large Cell Carcino (LCC), this weakness is attributed to insufficient number of LCC samples in the training dataset. The use of Depthwise Convolution (DwConv) within the Efficient Modulation presents a theoretical limitation by leading to lack of spatial correlation due to independent feature extraction across different channels.

Model	Faster R-CNN	YOLOv3	YOLOv5	YOLOv7	YOLOv8	YOLO-ED
P	0.798	0.823	0.850	0.845	0.879	0.907
R	0.802	0.827	0.822	0.858	0.871	0.904
mAP(@0.5)	0.819	0.843	0.851	0.865	0.873	0.905
GFLOPs	15.5	193.9	15.8	13.1	8.9	6.5
Params(M)	315.0	117.0	17.6	11.7	2.87	2.53

Table 4 Performance comparison of the different models on LUNG-PET-CT-DX.[5]

Model	Faster R-CNN	YOLOv3	YOLOv5	YOLOv7	YOLOv8	YOLO-ED
P	0.847	0.891	0.905	0.889	0.897	0.926
R	0.845	0.865	0.871	0.892	0.876	0.903
mAP(@0.5)	0.868	0.898	0.913	0.923	0.926	0.947
GFLOPs	15.5	193.9	15.8	13.1	8.9	6.5
Params(M)	315.0	117.0	17.6	11.7	2.87	2.53

Table 5 Performance comparison of the different models on LUNA16 [5]

2.3. Lessons Learned from Literatures

The comprehensive review of contemporary deep learning applications in pulmonary medical imaging reveals several key findings and underscores critical challenges that directly inform the architecture and methodology of high-performing automated diagnostic systems.

The literature review highlighted a triple imperative for success in medical AI: balancing real-time efficiency, high detection accuracy, and clinical interpretability.

1. Dominance of YOLO Architectures for Efficiency and Localization: The You Only Look Once (YOLO) architecture, including its advanced iterations (YOLOv7, YOLOv11), has emerged as the most efficient solution for real-time object detection and localization tasks (e.g., pneumonia, TB, and lung cancer detection). This efficiency stems from their ability to balance detection speed with computational complexity, making them practical for deployment in resource-constrained environments. The success of models like FAST-YOLO (which is 48 FPS faster than YOLOv11 on the MIMIC dataset) and YOLO-ED (which reduced computational load by 2.4 GFLOPs compared to YOLOv8) demonstrates the value of architectural modifications aimed at creating lightweight networks without sacrificing performance.
2. Addressing Data Imbalance and Scarcity: Deep learning models are inherently data-hungry, making data scarcity and class imbalance persistent challenges, particularly for rare or subtle manifestations (e.g., obsolete TB or Large Cell Carcinoma)

Chapter Three: Problem Analysis and Modeling

This chapter analyzes the context and requirements for the proposed AI-driven lung disease detection system. We first examine Ethiopia's existing radiology workflow and its shortcomings. Based on this, we derive the system's functional and non-functional requirements from national needs and the literature. Finally, we outline the system model through UML and data-flow diagram to illustrate behavior and structure.

3.1 Existing System and Its Problems

Ethiopia's current diagnostic process for lung diseases is largely manual and centralized. Few trained radiologists are available (roughly 300 radiologists for ~118 million people [6]), so most X-ray and CT scans must be interpreted by overburdened specialists. This shortage causes long delays: patients often wait days or weeks for reports, and many must travel from rural areas to urban centers for imaging [6]. In remote regions, access is worse – basic imaging equipment and specialists are scarce [7]. Consequently, many cases of TB, pneumonia, or lung cancer go undetected or are diagnosed very late, worsening outcomes.

These problems have broad impacts. Patients face delayed diagnoses and treatment, travel long distances, and incur higher costs. Clinicians struggle with high caseloads and wait for images and reports. Hospital administrators cannot easily share patient scans across branch facilities or ensure continuity of care. For example, without a unified system, a patient's prior X-ray from one clinic is not accessible at another clinic in the same network. Moreover, Ethiopia's health system has limited digital infrastructure: many hospitals still use film radiography or standalone X-ray machines. Digital health adoption is low, and there is heavy reliance on physical film transfer or siloed databases [6].

Some AI/CAD tools exist (for example WHO-supported TB-screening software), but they are disease-specific, standalone, and not integrated into a national workflow. In summary, the existing system suffers from staff shortages, poor rural coverage, fragmented information flow, and limited use of AI/CAD. These issues lead to long patient wait times, decreased satisfaction, and adverse health outcomes [6].

3.2. Specifying the Requirements of the Proposed Solution

To address these gaps, we define requirements for an AI-powered lung disease detection and decision-support system. Functional requirements enumerate the system's features (e.g. image upload, diagnosis, notifications), while non-functional requirements specify quality attributes (e.g. reliability, usability). Because no direct surveys or interviews were conducted, requirements are inferred from Ethiopia's healthcare context and best practices in the literature.

3.3. System Modeling

3.3.1. Functional Requirements

3.3.1.1. Authentication & User Management Requirements

FR-01 User Login

The system shall allow registered users (Doctor/Admin) to securely log in using email and password.

FR-02 Two-Factor Authentication

The system shall support two-factor authentication email OTP.

FR-03 Role Management

The system shall enforce different roles:

- Lab technican/Radiologist
- Doctor
- Admin

FR-04 Logout

Users shall be able to terminate the session securely at any time.

3.3.1.2. Image Upload & Validation Requirements

FR-05 Upload Chest X-ray

The lab technician/radiologist shall upload X-ray images in:

- PNG
- JPEG/JPG
- DICOM (.dcm)

FR-06 Validation of Format/Size

The system shall validate:

- File type
- pixel resolution
- Invalid or corrupted headers
- Acceptable size limit

FR-07 Duplicate Image Detection

The system shall detect duplicate images using hashing and notify the user.

3.3.1.3. AI Inference & Analysis Requirements

FR-08 Image Preprocessing

Before inference, the system shall:

- resize image
- normalize pixel values
- enhance contrast dynamically

FR-09 AI-Based Disease Detection (YOLOx Model)

The system shall classify abnormal regions into:

1. Pneumonia
2. Tuberculosis
3. Lung Thummer

FR-10 Generate Bounding Boxes

The system shall produce bounding box overlay for detected abnormalities.

FR-11 Class Probability Distribution

The system shall calculate and display confidence score for each detected region.

FR-12 Inference Result Storage

System shall store:

- raw image
- prediction objects
- bounding box coordinates
- inference metadata

3.3.1.4. Admin Radiologist and Doctor Workflow Requirements

FR-13 Review System Predictions

Radiologist shall view:

- bounding boxes
- prediction scores
- Add Note

FR-14 Edit Detection Annotation

Radiologist shall be able to:

- correct bounding regions
- remove false positives
- add missed detection areas
- modify assigned disease class

FR-15 Adjust Confidence Filters

Radiologist shall adjust threshold slider (0-100%).

FR-16 Final Diagnosis Submission

Doctor shall enter:

- diagnosis decision
- doctor notes
- urgency level (critical / non-critical)
- View X-ray images sent by the radiologist

FR-17 User role management

Admin should be able to modify and assign user's roles

FR-18 System Log monitiring

Admin must be able to review system Logs

FR-19 Diagnosis access

Admin can not access nor approve diagnosis

3.3.1.5. Reporting Requirements

FR-20 Download Report as PDF

Doctor shall download a formatted PDF containing full case summary.

FR-21 Generate Report

A report must contain:

- Original radiograph
- Annotated image
- Model findings
- Radiologist corrections
- Final decision and notes

FR-22 View Past Reports

Doctor shall view:

- All historical cases and diagnosis the doctor has handled.
- The patient history

3.3.1.6. Patient & Case Management Requirements

FR-23 Register New Patient Record

System shall store patient data:

- Age
- Sex
- Patient ID
- Visit date

FR-24 Case Linking

System shall allow linking follow-up visits with existing case.

FR-25 Patient-Based Search

Doctors shall search case history using:

- patient ID
- diagnosis type
- date range

.7 Administrative Requirements

FR-26 User Management

Admin shall:

- add users
- deactivate users
- reset credentials
- assign roles

FR-27 Access Log Viewing
Admin shall view system-wide access logs filtered by:

- action type
- case ID
- user ID

3.3.2. Non-Functional Requirements

3.3.2.1. Performance Requirements

NFR-1 Response Speed
End-to-end image analysis (from upload to inference to save) shall complete within ≤ **12 seconds** on a single GPU machine.

NFR-2 Interface Interaction Time
UI interactions such as toggling overlays shall render in less than **300 milliseconds**.

NFR-3 Report Generation Time
A full PDF medical report must generate within **≤ 10 seconds** if cached and ≤ 18 seconds if no cache exists.

NFR-4 Concurrent Session Support
System shall support **200 simultaneous logged-in users** without failure or slowdown.

3.3.2.2. Reliability & Availability Requirements

NFR-5 Uptime

System shall have **>= 95% availability** for production deployment.

NFR-6 Maintainace time

At minimum the system will need 3 hours per month to update new model weights.

NFR-7 Automatic Retry

Inference jobs shall retry once if failure occurs and then queue for admin review.

NFR-8 Automatic Recovery

Core services must restart automatically on server failure without manual intervention.

NFR-9 Scheduled Backups

Daily and weekly backups of:

- inference results
- Reports
- patient data
- System logs

must be performed.

NFR-10 Disaster Recovery

System must restore backups within 2 **hours**.

3.3.2.3. Security Requirements

NFR-11 Data Encryption

Required:

- TLS 1.2+ transmission (Through host services)
- AES-256 encryption for storage

NFR-12 Access Restrictions

Only Doctors may finalize diagnoses; Admin cannot approve diagnosis.

NFR-13 Audit Trail

System shall maintain immutable encrypted logs for:

- Logins
- X-ray image uploads
- Prediction
- When results are sent to Doctors
- diagnosis modifications
- Deletions

- report generation events

NFR-14 System Data Protection

No credentials, communications, logs, and diagnosis can exist as plaintexts in database or source code.

NFR-15 Easy Learning

A user must be able to complete the workflow (upload to review to finalize) within **10 minutes** of first usage.

NFR-16 Clear Visual Interface

The UI must provide:

- zoom controls
- annotation tools
- toggle between normal/detected view

NFR-17 Help Documentation

System must include:

- guided help
- Tooltips
- instructional onboarding pop-ups

3.3.2.4. Maintainability Requirements

NFR-18 Component Isolation

AI inference, UI client, backend API, and reporting modules shall be independently updatable.

NFR-19 CI/CD Pipeline

Software updates must occur via automated deployment pipelines.

NFR-20 Code Documentation

Every module must include:

- API documentation
- input/output specs

3.3.2.5. Compatibility Requirements

NFR-21 Browser Support

Platform must support latest and previous versions of:

- Chrome
- Firefox
- Edge
- Safari

NFR-22 API Interoperability

Expose REST endpoints supporting JSON responses.

3.3.2.6. Accessibility Requirements

NFR-23 WCAG Compliance

System must meet WCAG 2.1 AA compliance guidelines.

NFR-24 Screen Reader Support

Labeling shall exist for:

- diagnostic icons
- annotated regions

3.3.2.7. Privacy Requirements

NFR-25 Patient Data Minimization

Only clinically essential information shall be stored.

NFR-26 Consent Processing

System shall record consent acknowledgment for each uploaded case.

NFR-27 Right to Erasure

System must provide deletion workflow.

3.3.2.8. Localization & Cultural Relevance Requirements

NFR-28 Language Support

System shall support at minimum:

- English
- Amharic

NFR-29 Locale-Based Formats

Date/time must be formatted based on region (with timezone support).

3.3.3. Class Diagram

The class diagram defines the static structural model of the system, identifying the main classes, their attributes, operations, and the relationships among them. It provides a clear representation of how system entities such as hospitals, users, patients, medical cases, images, inference results, reviews, diagnoses, reports, and audit logs are organized and interact within the system. This model supports the functional and non-functional requirements by formalizing data ownership, role-based responsibilities, lifecycle states, and information flow. Additionally, the use of enumerations ensures consistency in system states and domain-specific values, facilitating maintainability, traceability, and alignment with the requirements specified in this document.

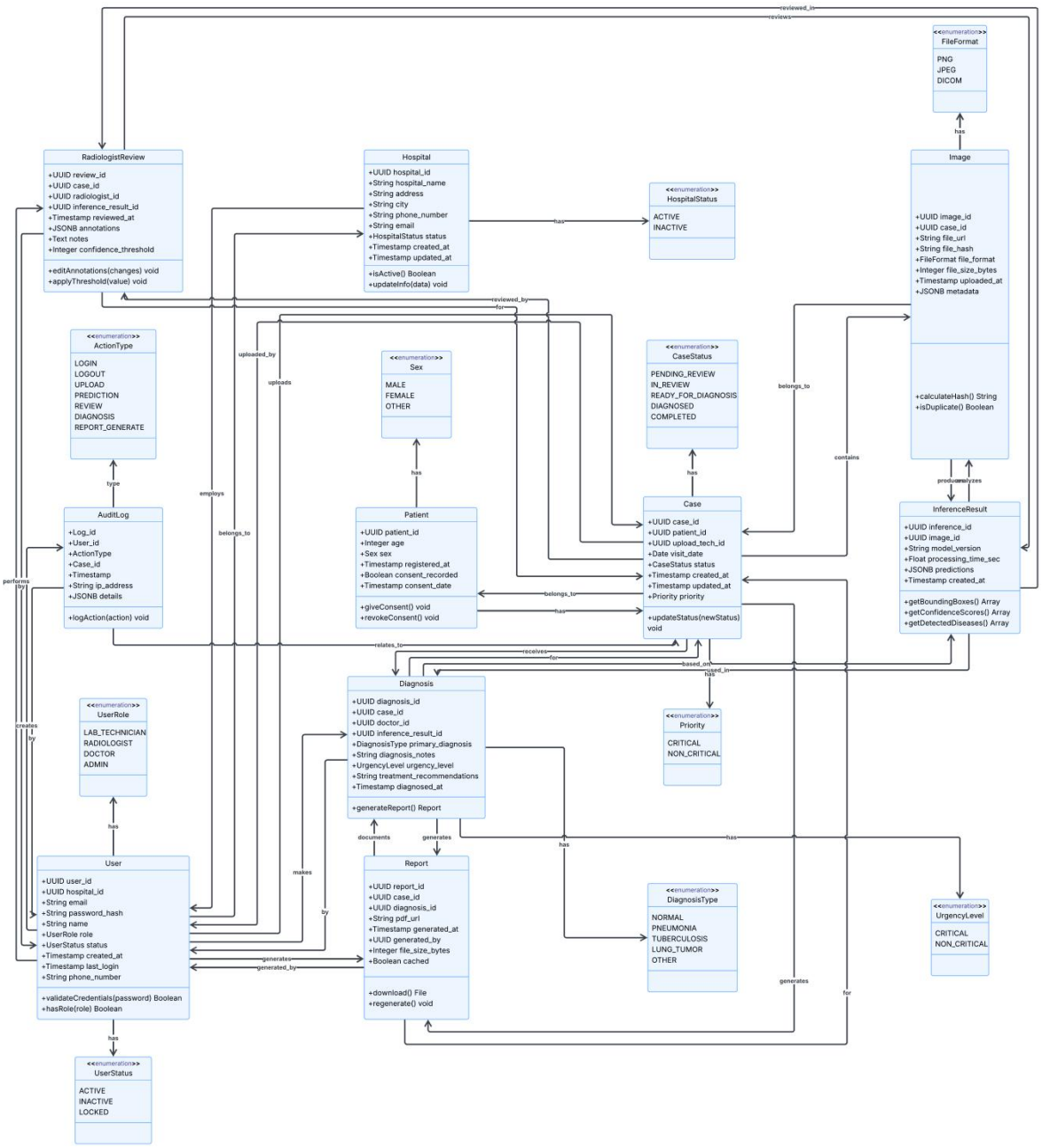

Image 2 Class Diagram

3.3.4. Usecase

The Use Case Diagram (Figure X) provides a high-level view of the functional requirements for the X-ray Analysis System, illustrating the interactions between external actors and the system's internal modules. The system boundary encompasses seven distinct functional packages, ranging from core image processing to administrative oversight, ensuring a clear separation of concerns.

Three primary actors interact with the system, each with specific privileges:

- Lab Technicians/Radiologists: Responsible for the initial data ingestion and quality control. They upload X-ray images, validate quality, review AI-generated predictions, and refine annotations (e.g., bounding boxes) before final diagnosis.
- Doctors: Computed with the final clinical decision-making authority. They utilize the system to view X-rays, submit final diagnoses (with urgency levels), generate comprehensive PDF reports, and manage patient records.
- Administrators: Tasked with system maintenance and security. Their role focuses on user management, role assignments, and audit log monitoring. Notably, the diagram highlights a critical security constraint: administrators are explicitly restricted from accessing sensitive diagnosis data to maintain HIPAA compliance.

The core workflow is automated through dependency relationships (triggered/included). The process initiates with Image Upload, which automatically triggers validation and duplicate detection. Successful uploads flow into the AI Inference pipeline, where the system performs preprocessing and executes the YOLOx model for disease detection (Pneumonia, TB, Tumors). The results of this inference enable the subsequent review and reporting phases, culminating in the generation of a downloadable diagnosis report.

Actor	Description
Lab Technician/ Radiologist	Responsible for uploading and validating chest X-ray or CT images, ensuring they meet the required format and quality standards before processing. Reviews AI-generated annotations, edits bounding boxes, and adjusts disease classifications as needed, providing expert oversight to enhance diagnostic accuracy.
Doctor	Finalizes the diagnosis based on AI and radiologist inputs, adds clinical notes, and generates comprehensive PDF reports for

	patient records.
Admin	Manages user accounts, assigns roles, and audits system logs to ensure secure and compliant operations across the platform.

Table 6 Usecase diagram

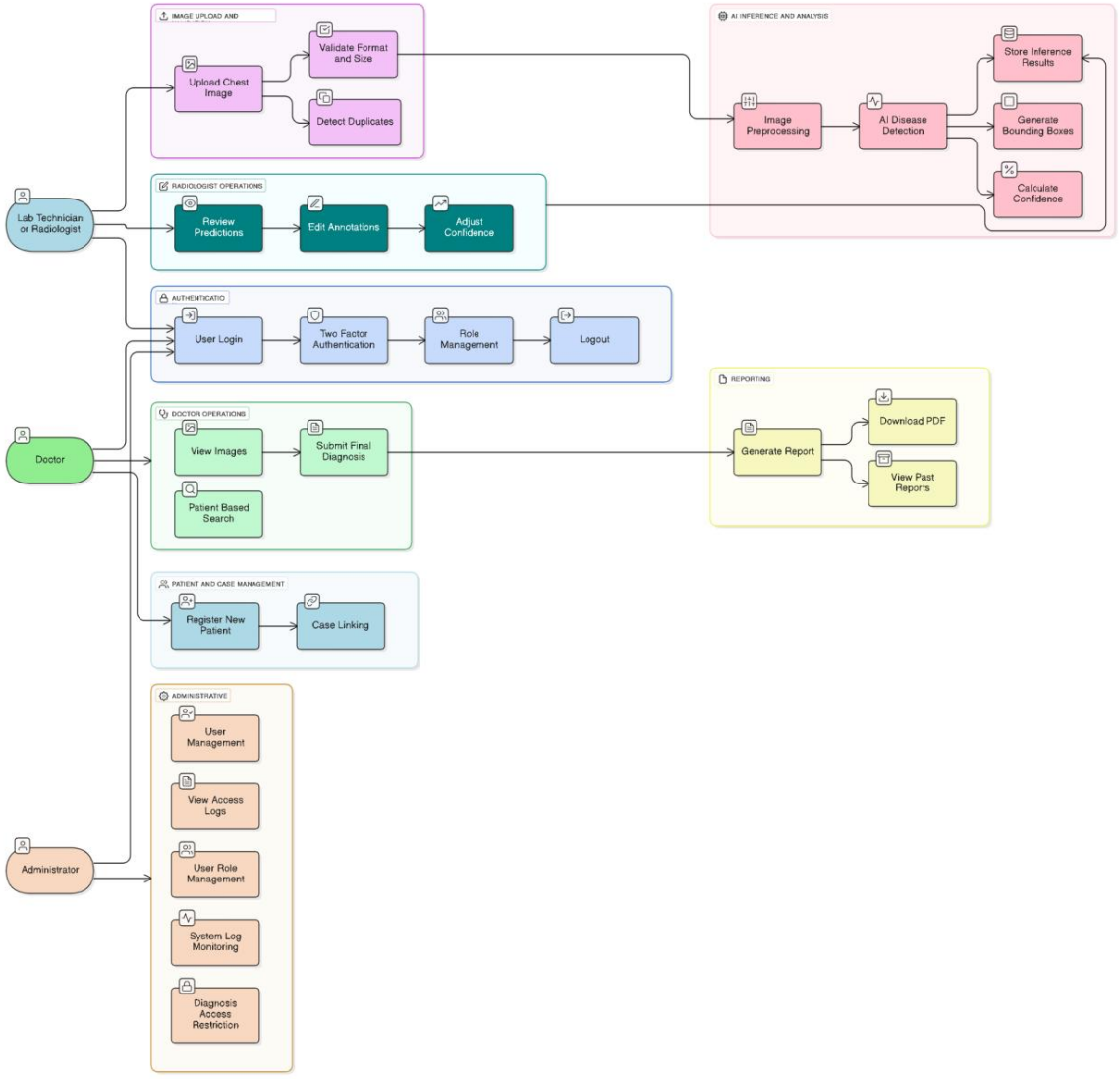

Image 3 Usecase Diagram

Use case name	User Login
Use case ID	UC_01

Actors	Lab Technician/Radiologist, Doctor, Administrator
Use case description	Description: Registered users securely authenticate to access the system using email and password credentials. Trigger: User navigates to the login page and attempts to access the system. Precondition: • User has a registered account with valid credentials • System is operational and accessible Post-condition: • User is authenticated and granted access based on their role • Session is created and logged in the system • User is redirected to their role-specific dashboard Basic Flow: 1. User enters email and password on the login page 2. System validates the credentials against the database 3. System verifies user role (Lab Technician/Radiologist, Doctor, or Admin) 4. System creates a secure session token 5. System redirects user to appropriate dashboard based on role 6. System logs the successful login attempt Alternate Flow: 1. Invalid credentials: System displays error

	message and allows retry (max 3 attempts) 2. Account locked: System notifies user and suggests password reset 3. System error: Display maintenance message and log the error

Table 7 UC_01

Use case name	User Logout
Use case ID	UC_02
Actors	Lab Technician/Radiologist, Doctor, Administrator
Use case description	Description: User securely terminates their active session and logs out of the system. Trigger: User clicks the logout button or session times out. Precondition: • User has an active authenticated session Post-condition: • Session is terminated and invalidated • User is redirected to login page • Logout event is logged Basic Flow: 1. User clicks the logout button 2. System confirms logout action 3. System invalidates the session token 4. System clears any cached sensitive data 5. System redirects user to login page 6. System logs the logout event Alternate Flow:

	1. Auto-logout due to inactivity timeout (30 minutes) 2. System displays session expired message and redirects to login

Table 8 UC_02

Use case name	Upload Chest X-ray
Use case ID	UC_03
Actors	Lab Technician/Radiologist
Use case description	Description: Lab Technician uploads chest X-ray images in supported formats (PNG, JPEG/JPG, DICOM) for analysis. Trigger: Lab Technician selects and uploads an X-ray image file. Precondition: • Lab Technician is authenticated and logged in • Patient record exists or can be created • Image file is in supported format (PNG/JPEG/DICOM) Post-condition: • Image is successfully uploaded to the system • Image is stored in the database • Upload event is logged • Validation process is triggered Basic Flow: 1. Lab Technician navigates to image upload page 2. Lab Technician selects patient record or creates new one 3. Lab Technician clicks "Upload Image" button

	4. System displays file selection dialog 5. Lab Technician selects X-ray image file 6. System displays upload progress 7. System confirms successful upload 8. System automatically triggers validation Alternate Flow: 1. File size exceeds limit: System displays error and rejects upload 2. Network interruption: System resumes upload from last checkpoint 3. Unsupported format: System displays format error and lists accepted formats

Table 9 UC_03

Use case name	Review System Predictions
Use case ID	UC_04
Actors	Lab Technician/Radiologist
Use case description	Description: Radiologist reviews AI-generated predictions, including bounding boxes, confidence scores, and can add notes for the Doctor. Trigger: Radiologist accesses a case with completed AI inference or receives notification. Precondition: • Radiologist is authenticated and logged in • AI inference results are available • Case is assigned or accessible to Radiologist Post-condition: • Radiologist has viewed all AI predictions

	• Any notes added are saved to case • Review timestamp is logged • Case status updated to "Reviewed" Basic Flow: 1. Radiologist navigates to pending cases dashboard 2. Radiologist selects a case for review 3. System displays original X-ray image 4. System overlays bounding boxes and labels 5. Radiologist views confidence scores for each detection 6. Radiologist examines classification labels 7. Radiologist adds review notes (optional) 8. Radiologist can proceed to edit annotations or send to Doctor 9. System logs review completion Alternate Flow: 1. No detections found: Radiologist reviews image manually and can add annotations 2. Disagreement with AI: Radiologist proceeds to edit annotations 3. Request second opinion: System allows flagging case for peer review

Table 10 UC_04

Use case name	Edit Detection Annotations
Use case ID	UC_05
Actors	Lab Technician/Radiologist
Use case description	Description: Radiologist refines AI-generated

	annotations by correcting bounding boxes, removing false positives, adding missed detections, and modifying disease classifications. Trigger: Radiologist determines AI predictions need correction during review . Precondition: • Radiologist is reviewing a case • Annotation editing tools are available • Original AI predictions are loaded Post-condition: • Annotations are updated with radiologist corrections • Original AI predictions are preserved for comparison • Modified annotations are stored • Edit history is logged with timestamp and user ID Basic Flow: 1. Radiologist clicks "Edit Annotations" button 2. System enables annotation editing mode 3. Radiologist selects bounding box to modify 4. Radiologist adjusts box position/size by dragging corners 5. Radiologist changes disease classification if needed 6. Radiologist removes false positive detections 7. Radiologist adds new bounding boxes for missed regions 8. Radiologist assigns disease class to new regions 9. Radiologist saves modifications 10. System updates stored results and logs

	changes Alternate Flow: 1. Discard changes: Radiologist can cancel and revert to AI predictions 2. Add entirely new detection: Radiologist draws new bounding box and assigns class 3. Remove all detections: Radiologist marks case as "No abnormalities found"

Table 11 UC_05

Use case name	Adjust Confidence Filters
Use case ID	UC_06
Actors	Lab Technician/Radiologist
Use case description	Description: Radiologist adjusts confidence threshold slider to filter which AI detections are displayed based on probability scores. Trigger: Radiologist wants to filter detections by confidence level during review. Precondition: • Radiologist is reviewing a case with AI predictions • Multiple detections with varying confidence scores exist Post-condition: • Display is filtered to show only detections meeting threshold • Filter setting is saved for session • Lower confidence detections are hidden but not deleted Basic Flow:

	1. Radiologist accesses confidence filter control 2. System displays current threshold (default 50%) 3. Radiologist adjusts slider between 0-100% 4. System dynamically updates detection display 5. System shows/hides detections based on threshold 6. System displays count of visible vs. total detections 7. Radiologist views filtered results 8. System saves filter preference for session Alternate Flow: 1. Show all: Radiologist sets threshold to 0% to view all detections 2. High confidence only: Radiologist sets threshold to 80%+ for critical cases 3. Reset to default: Radiologist clicks reset to return to 50% threshold

Table 12 UC_06

Use case name	Submit Final Diagnosis
Use case ID	UC_07
Actors	Doctor
Use case description	Description: Doctor reviews radiologist-validated images and AI predictions, then submits final clinical diagnosis with notes and urgency level. Trigger: Doctor accesses a case ready for final diagnosis. Precondition:

	• Doctor is authenticated and logged in • Case has completed radiologist review • X-ray images and predictions are available Post-condition: • Final diagnosis is recorded in system • Diagnosis notes and urgency level are stored • Case status updated to "Diagnosed" • Report generation is triggered • Diagnosis event logged in audit trail Basic Flow: 1. Doctor navigates to pending diagnosis queue 2. Doctor selects a case for diagnosis 3. System displays X-ray images with radiologist annotations 4. Doctor reviews AI predictions and radiologist notes 5. Doctor examines patient history if available 6. Doctor enters final diagnosis decision 7. Doctor adds clinical notes and recommendations 8. Doctor selects urgency level (Critical/Non-Critical) 9. Doctor submits diagnosis 10. System validates and saves diagnosis 11. System triggers report generation 12. System logs diagnosis event with timestamp Alternate Flow: 1. Insufficient information: Doctor requests additional images or tests 2. Second opinion needed: Doctor flags case for

	peer consultation 3. Critical case: System immediately notifies relevant stakeholders

Table 13 UC_07

Use case name	View X-ray Images
Use case ID	UC_08
Actors	Doctor
Use case description	Description: Doctor views X-ray images sent by radiologist with AI annotations and radiologist modifications. Trigger: Doctor selects a case to review for diagnosis. Precondition: • Doctor is authenticated and logged in • Images have been uploaded and processed • Case is accessible to doctor Post-condition: • Doctor has viewed all relevant images • Image viewing activity is logged • Doctor can proceed to diagnosis Basic Flow: 1. Doctor opens case from dashboard 2. System displays original X-ray image 3. Doctor toggles between original and annotated views 4. Doctor uses zoom controls to examine details 5. Doctor views bounding boxes and

	classifications 6. Doctor reads radiologist notes 7. Doctor examines confidence scores 8. Doctor compares multiple images if available Alternate Flow: 1. Image quality issues: Doctor can request re-upload 2. Multiple views: Doctor switches between different X-ray angles/dates

Table 14 UC_08

Use case name	Patient-Based Search
Use case ID	UC_09
Actors	Doctor
Use case description	Description: Doctor searches and retrieves case history using patient ID, diagnosis type, or date range filters. Trigger: Doctor needs to find specific patient cases or historical records. Precondition: • Doctor is authenticated and logged in • Patient records exist in database • Search functionality is operational Post-condition: • Relevant cases are retrieved and displayed • Search results show matching records • Search query is logged Basic Flow:

	1. Doctor navigates to search interface 2. Doctor enters search criteria (Patient ID/Diagnosis/Date) 3. Doctor applies optional filters 4. System queries database with search parameters 5. System retrieves matching cases 6. System displays results in list format 7. Doctor selects a case to view details 8. System logs search activity Alternate Flow: 1. No results found: System displays "No matching cases" message 2. Multiple filters: Doctor combines patient ID + date range for precision 3. Export results: Doctor can export search results for external analysis

Table 15 UC_09

Use case name	Generate Report
Use case ID	UC_10
Actors	Doctor
Use case description	Description: System automatically generates comprehensive medical report containing original radiograph, annotated image, AI findings, radiologist corrections, and final diagnosis. Trigger: Doctor submits final diagnosis or manually requests report generation. Precondition:

	• Final diagnosis has been submitted • All case data is complete and accessible • Report template is available Post-condition: • Comprehensive PDF report is generated • Report meets requirements • Report is stored in system • Report generation completes within ≤18 seconds • Report is available for download Basic Flow: 1. System receives report generation trigger 2. System retrieves all case data from database 3. System loads report template 4. System embeds original X-ray image 5. System embeds annotated image with bounding boxes 6. System includes AI model findings and confidence scores 7. System includes radiologist corrections and notes 8. System includes final diagnosis and doctor notes 9. System adds patient information (Age, Sex, ID, Visit Date) 10. System adds urgency level and recommendations 11. System generates PDF document 12. System stores report in database 13. System notifies Doctor of completion

	Alternate Flow: 1. Template error: System logs error and uses backup template 2. Image embedding fails: System retries or includes image reference 3. Generation timeout: System queues for background processing

Table 16 UC_10

Use case name	Download Report as PDF
Use case ID	UC_11
Actors	Doctor
Use case description	Description: Doctor downloads the generated PDF medical report containing full case summary for patient records. Trigger: Doctor requests to download report after generation. Precondition: • Report has been generated • Doctor has access rights to the case • PDF file is accessible in storage Post-condition: • PDF report is downloaded to doctor's device • Download event is logged in audit trail • Report remains available in system Basic Flow: 1. Doctor views case with generated report 2. Doctor clicks "Download PDF Report" button 3. System verifies doctor's access rights

	4. System retrieves PDF from storage 5. System initiates file download 6. Browser saves PDF to doctor's downloads folder 7. System logs download event with timestamp and user ID Alternate Flow: 1. Report not yet generated: System offers to generate report first 2. Download interrupted: Doctor can retry download 3. Preview before download: System displays PDF in browser viewer

Table 17 UC_11

Use case name	View Past Reports
Use case ID	UC_12
Actors	Doctor
Use case description	Description: Doctor views all historical cases and diagnoses they have handled, including complete patient history. Trigger: Doctor navigates to past reports/history section. Precondition: • Doctor is authenticated and logged in • Historical diagnosis records exist • Doctor has previously finalized diagnoses Post-condition: • List of past cases is displayed

	• Doctor can access individual case details • View activity is logged Basic Flow: 1. Doctor navigates to "Past Reports" section 2. System queries all cases handled by doctor 3. System retrieves patient history records 4. System displays cases in chronological order 5. Doctor can filter by date range, patient, or diagnosis type 6. Doctor selects a case to view details 7. System displays full case information and report 8. Doctor can re-download previous reports Alternate Flow: 1. No past reports: System displays "No historical cases found" 2. Large result set: System implements pagination (25 per page) 3. Export history: Doctor can export list to CSV for external records

Table 18 UC_12

Use case name	Register New Patient Record
Use case ID	UC_13
Actors	Doctor
Use case description	Description: System stores new patient demographic data including Age, Sex, Patient ID, and Visit Date for case management.

	Trigger: New patient case is initiated or uploaded X-ray has no linked patient. Precondition: • User is authenticated (Doctor or Lab Technician) • Patient ID is unique and not already in system • Consent is obtained Post-condition: • New patient record is created in database • Patient data is encrypted • Record is available for case linking • Registration event is logged Basic Flow: 1. User clicks "Register New Patient" 2. System displays patient registration form 3. User enters Patient ID 4. User enters Age and Sex 5. User enters Visit Date 6. User records consent acknowledgment 7. User submits registration 8. System validates all required fields 9. System checks Patient ID uniqueness 10. System encrypts patient data 11. System saves to database 12. System confirms successful registration 13. System logs registration event Alternate Flow: 1. Duplicate Patient ID: System alerts user and suggests linking to existing record

	2. Missing required fields: System highlights errors and prevents submission 3. Invalid data format: System displays validation errors

Table 19 UC_13

Use case name	User Management
Use case ID	UC_14
Actors	Administrator
Use case description	Description: Administrator manages system users by adding new users, deactivating accounts, resetting credentials, and assigning roles. Trigger: Administrator needs to perform user account operations. Precondition: • Administrator is authenticated and logged in • Admin has user management privileges • User database is accessible Post-condition: • User accounts are created, modified, or deactivated • Role assignments are updated • User management actions are logged in audit trail • Affected users are notified of changes Basic Flow: Add User: 1. Admin navigates to User Management section 2. Admin clicks "Add New User"

	3. Admin enters user details (name, email) 4. Admin assigns role (Lab Technician/Radiologist/Doctor/Admin) 5. System validates email uniqueness 6. System creates account with encrypted credentials 7. System sends welcome email with registration link 8. System logs user creation Deactivate User: 1. Admin selects user from list 2. Admin clicks "Deactivate Account" 3. System confirms deactivation 4. System disables login for user 5. System logs deactivation with reason Reset Credentials: 1. Admin selects user 2. Admin clicks "Reset Password" 3. System generates temporary password 4. System sends reset email to user 5. System logs credential reset Assign Roles: 1. Admin selects user 2. Admin modifies role assignment 3. System updates permissions 4. System logs role change Alternate Flow: 1. Duplicate email: System rejects and displays error

	2. Deactivate own account: System prevents self-deactivation 3. Last admin user: System prevents deactivation to ensure admin access

Table 20 UC_14

Use case name	User Role Management
Use case ID	UC_15
Actors	Administrator
Use case description	Description: Administrator modifies and assigns user roles to control system access and permissions. Trigger: Administrator needs to change a user's role or permissions. Precondition: • Administrator is authenticated and logged in • Target user account exists • Role system is operational Post-condition: • User role is updated in system • User permissions are modified accordingly • Role change is logged • User is notified of role change Basic Flow: 1. Admin navigates to User Role Management 2. Admin searches for target user 3. System displays current user role 4. Admin selects new role from dropdown 5. System displays permission differences

	6. Admin confirms role change 7. System updates user role in database 8. System adjusts user permissions 9. System sends notification email to user 10. System logs role modification Alternate Flow: 1. No role change: Admin cancels operation 2. Promote to Admin: System requires secondary admin approval 3. Active session exists: System terminates user session for role refresh

Table 21 UC_15

3.3.5. Dynamic Models of a System

3.3.5.1. Sequence Diagrams

3.3.5.1.1. Image Upload and AI Analysis Workflow

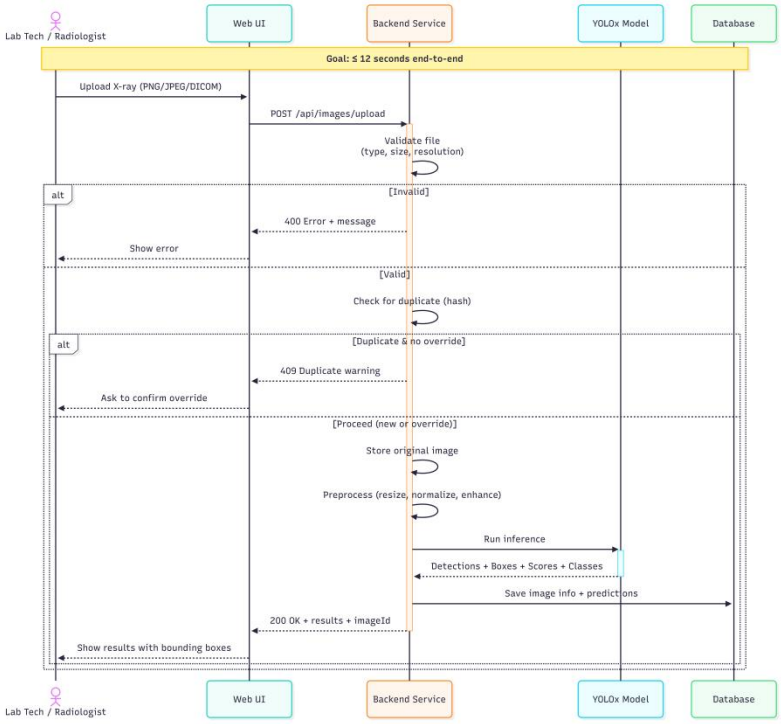

Image 4 Image Upload and AI Analysis Workflow

3.3.5.1.2. Radiologist Review and Annotation Workflow

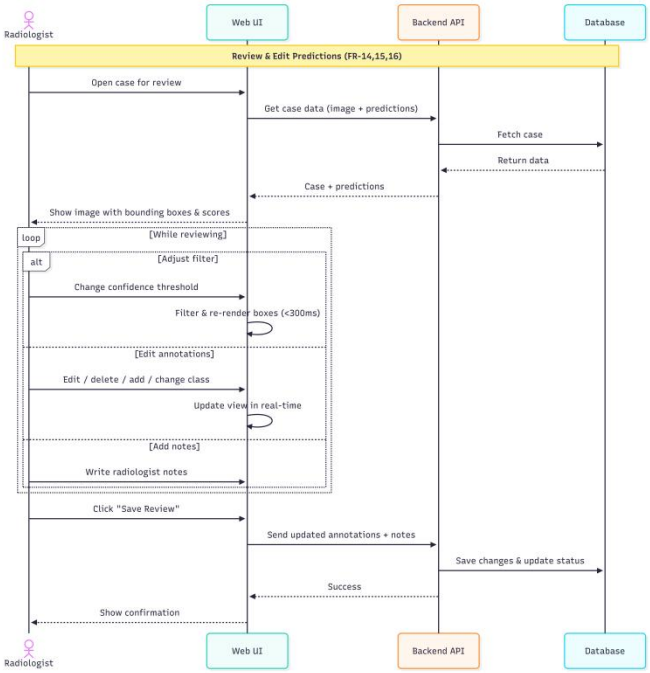

Image 5 Radiologist Review and Annotation Workflow

3.3.5.1.3. Doctor Diagnosis and Report Generation

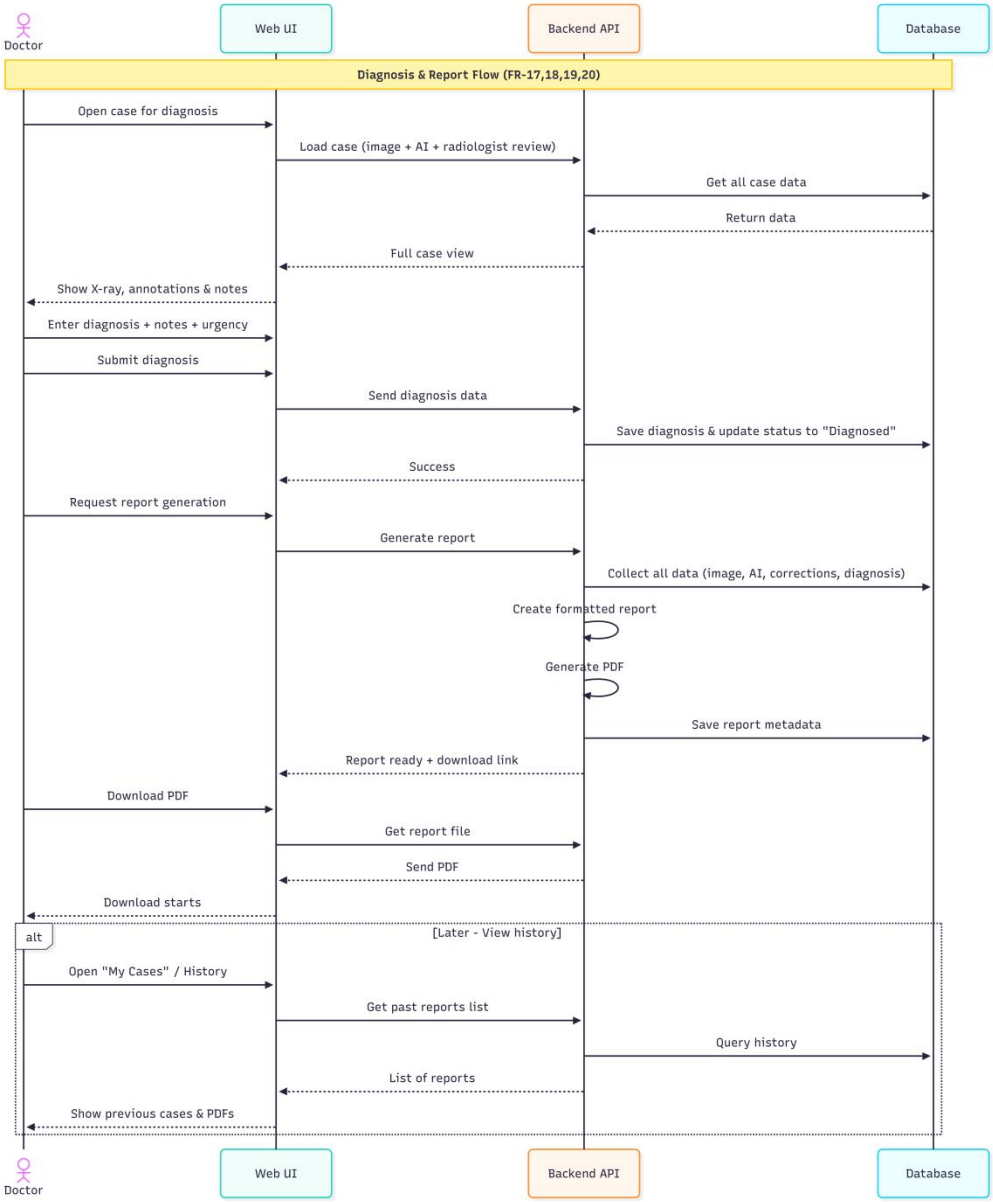

Image 6 Doctor Diagnosis and Report Generation

3.3.5.1.4. Admin User Management Workflow

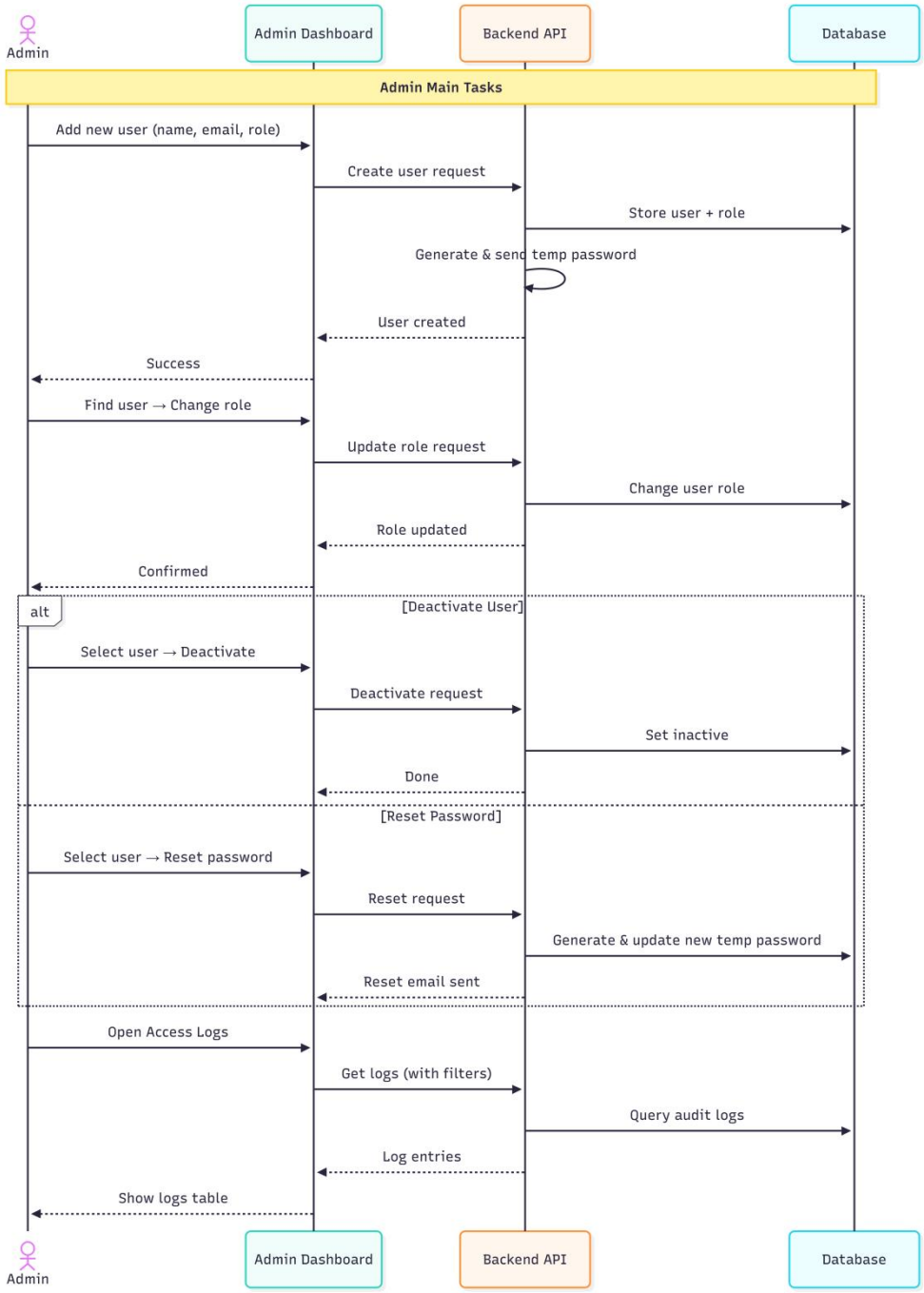

Image 7 Admin User Management Workflow

3.3.5.2. State Machine Diagrams

3.3.5.2.1. X-ray Image Upload & Processing

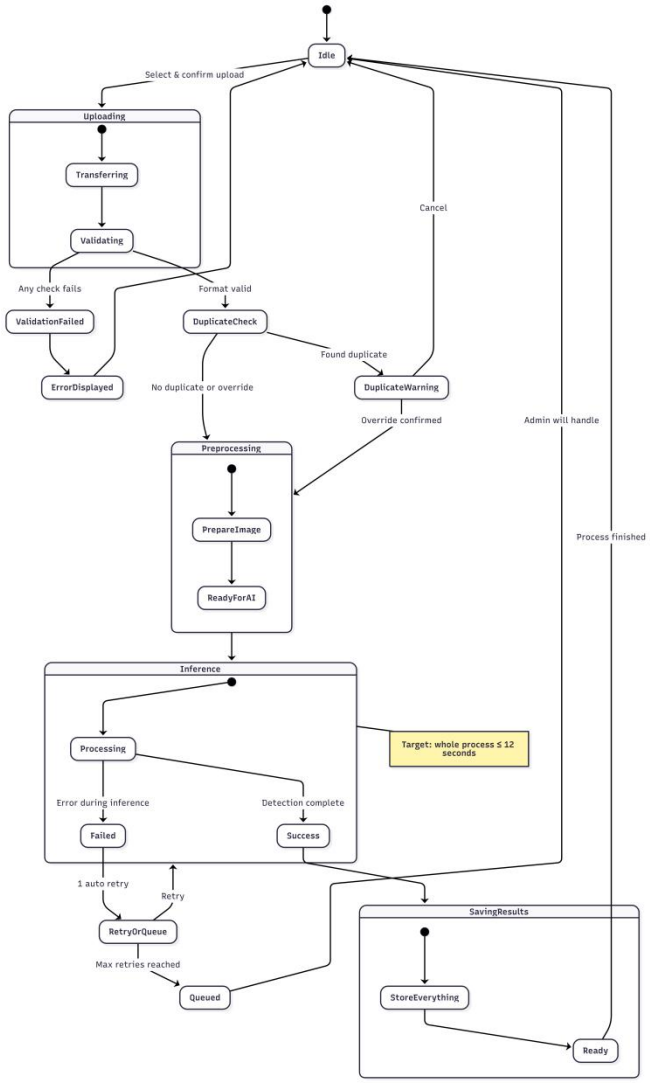

Image 8 X-ray Image Upload & Processing

3.3.5.2.2. Case or Diagnosis Lifecycle

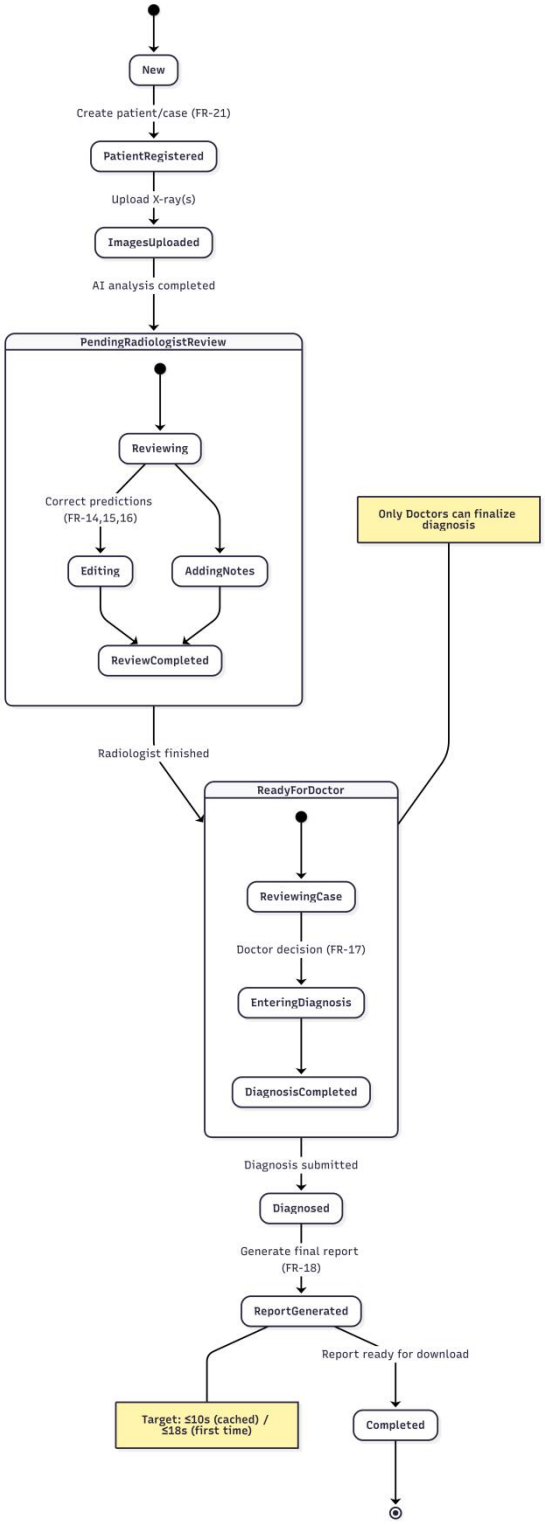

Image 9 Case or Diagnosis Lifecycle

3.3.5.2.3. User Account State Diagram

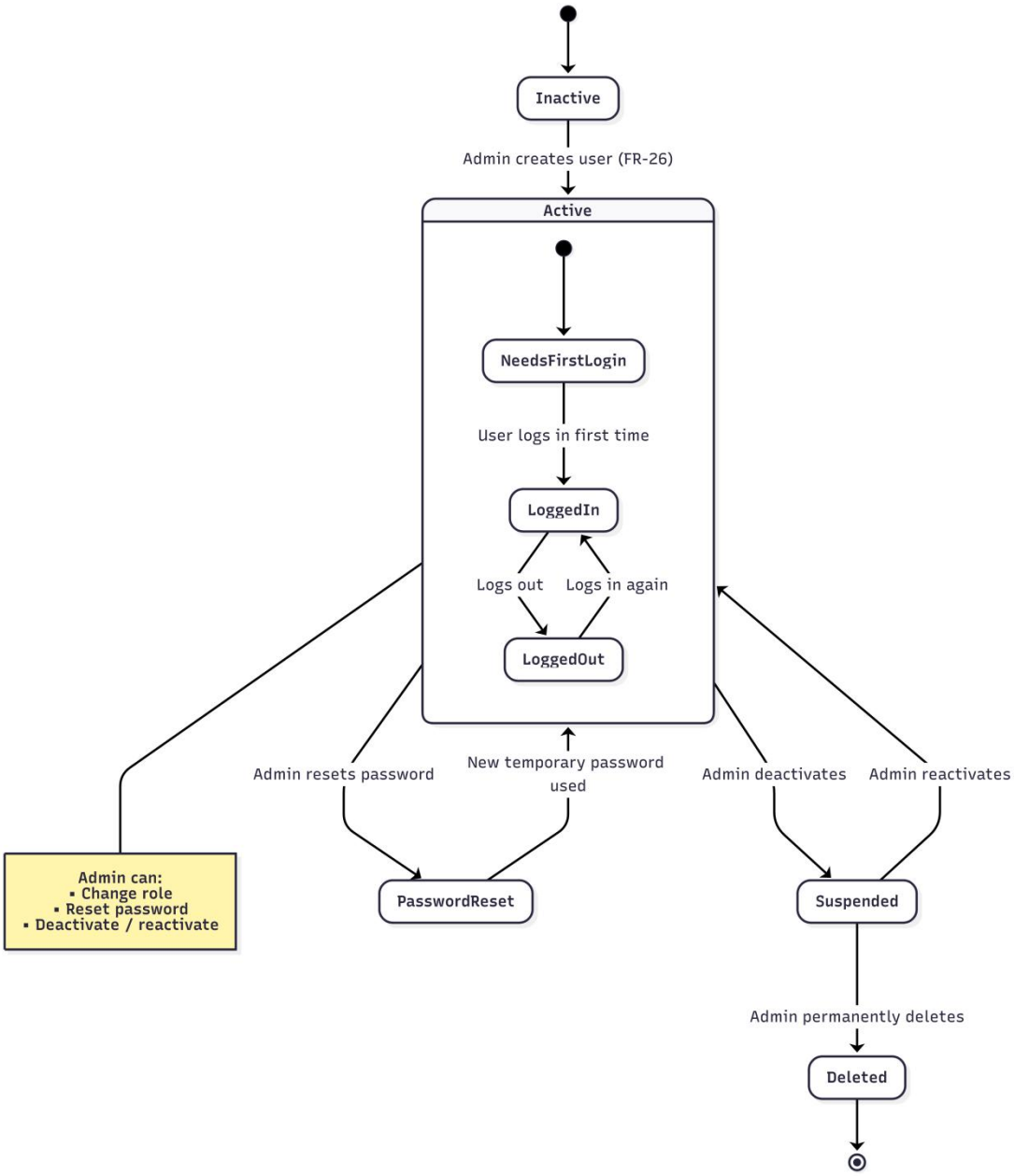

Image 10 User Account State Diagram

3.3.5.2.4. Image Analysis Request

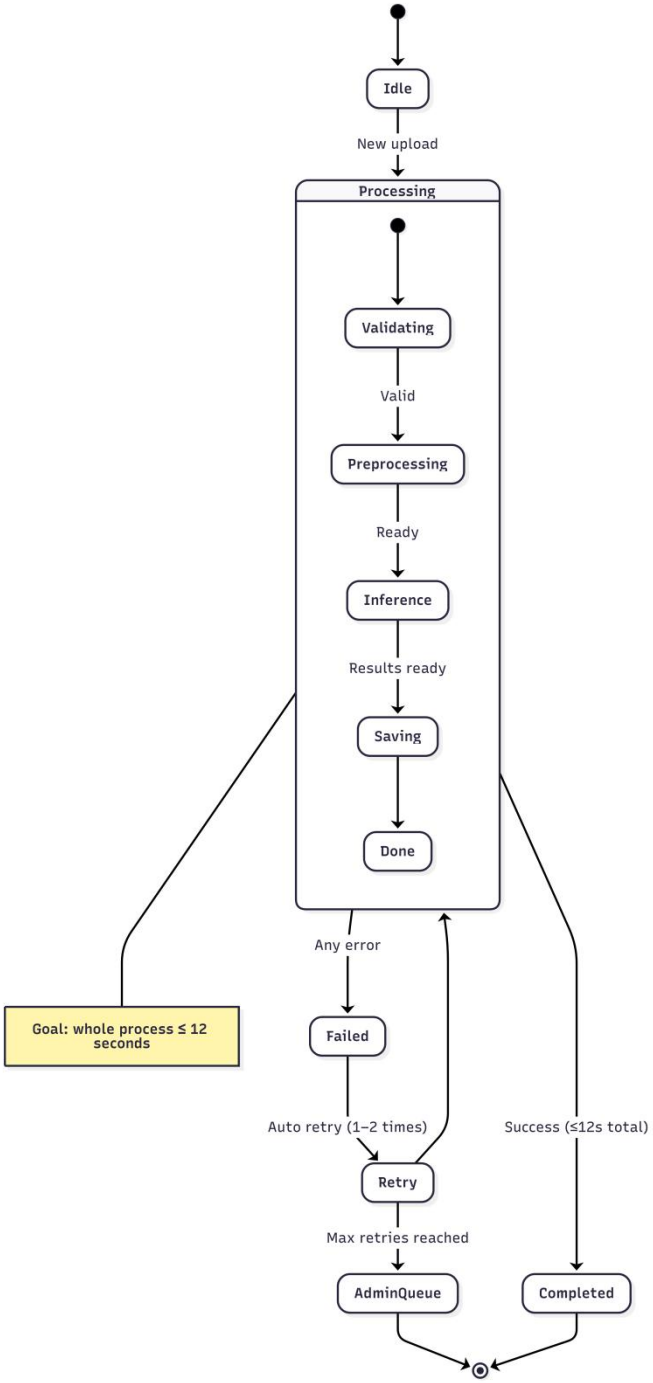

Image 11 Image Analysis Request

3.3.5.3 Activity Diagrams

3.3.5.3.1 Image Upload and AI Analysis Workflow

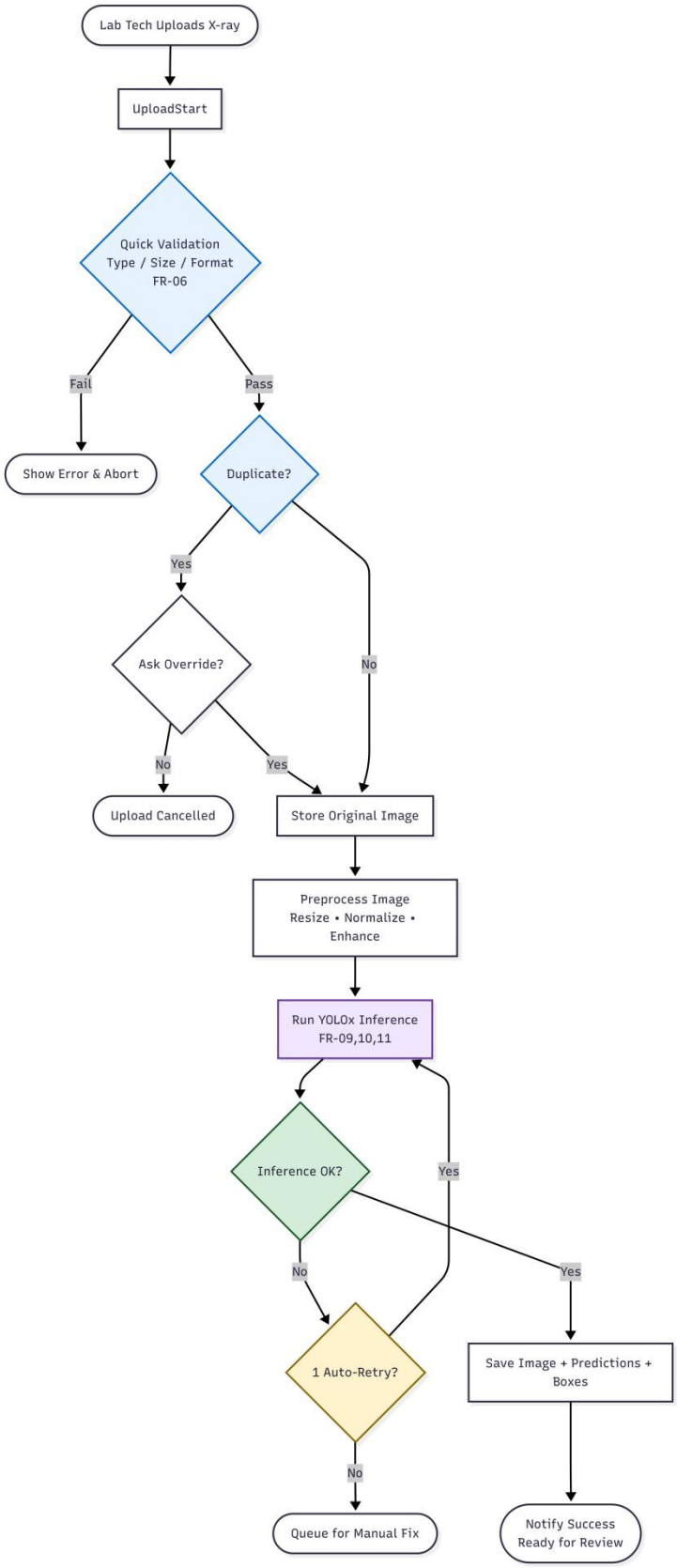

Image 12 Image Upload and AI Analysis Workflow

3.3.5.3.2 Radiologist Review and Annotation Activity

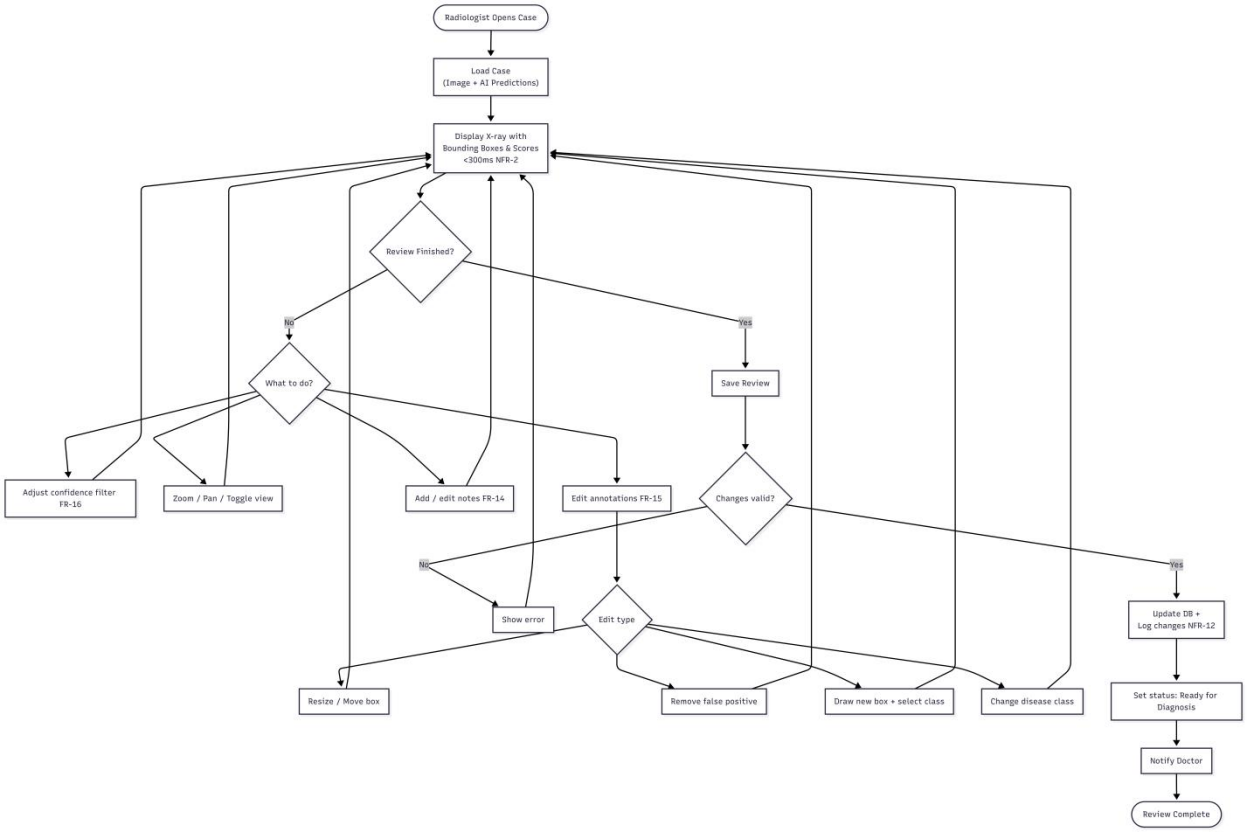

Image 13 Radiologist Review and Annotation Activity

3.3.5.3.3 Doctor Diagnosis and Report Generation

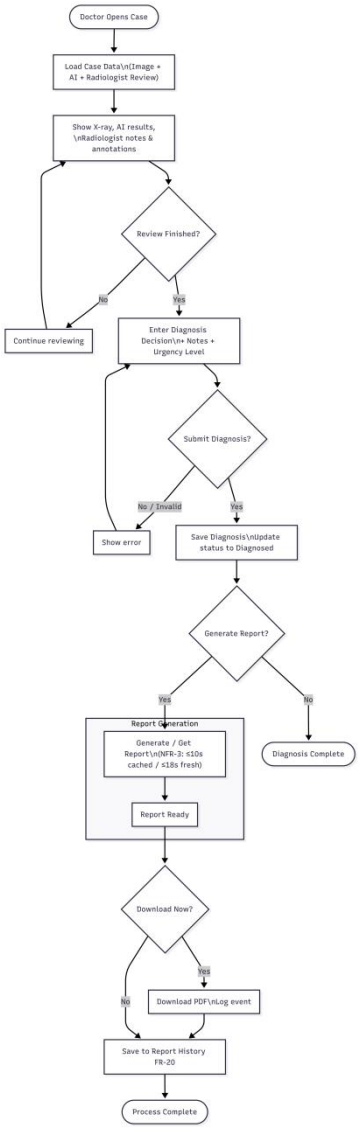

Image 14 Doctor Diagnosis and Report Generation

3.3.5.3.4 Admin User Management Activity

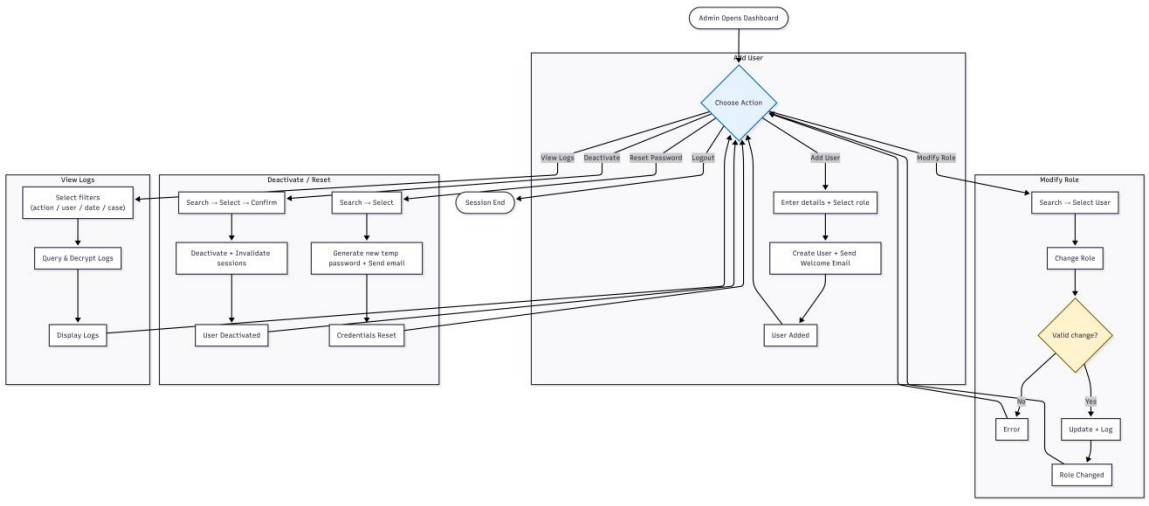

Image 15 Admin User Management Activity

3.3.5.4. Collaboration Diagrams

3.3.5.4.1 Image Upload and AI Analysis Collaboration

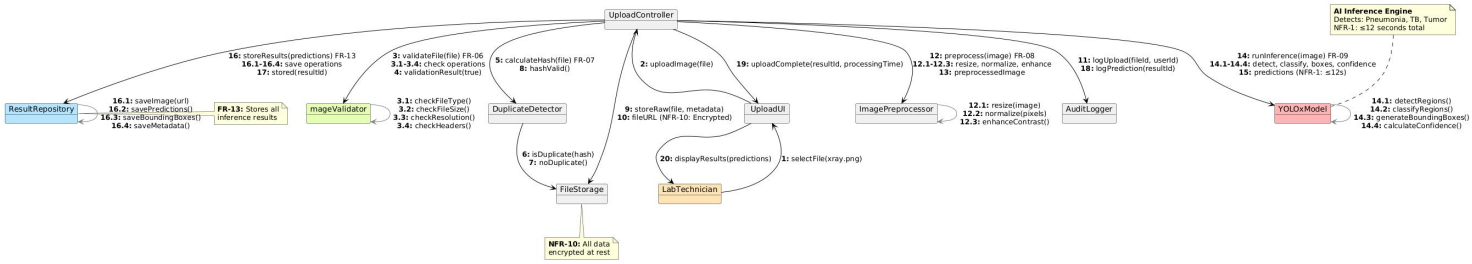

Image 16 Image Upload and AI Analysis Collaboration

3.3.5.4.2 Radiologist Review and Annotation Collaboration

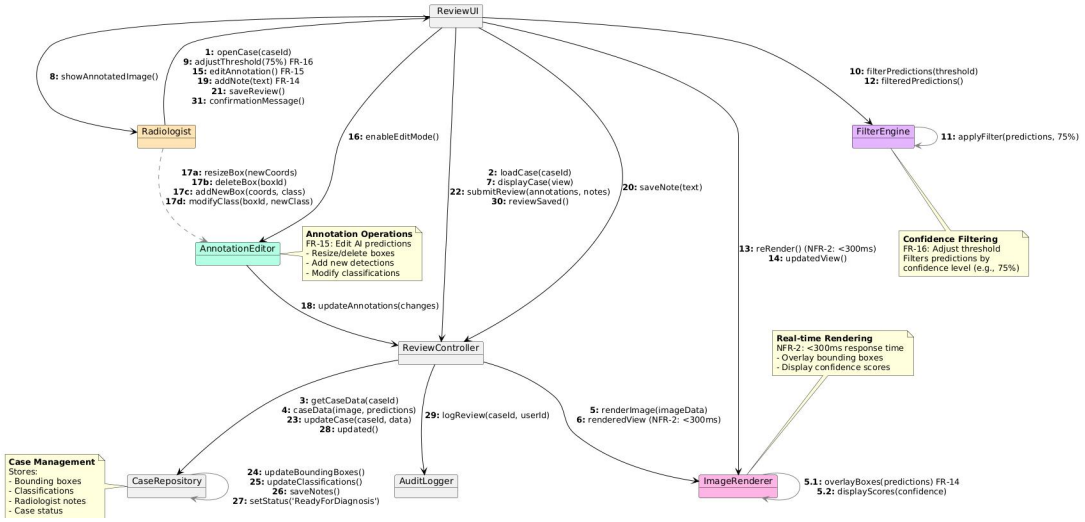

Image 17 Radiologist Review and Annotation Collaboration

3.3.5.4.3 Doctor Diagnosis and Report Generation Collaboration

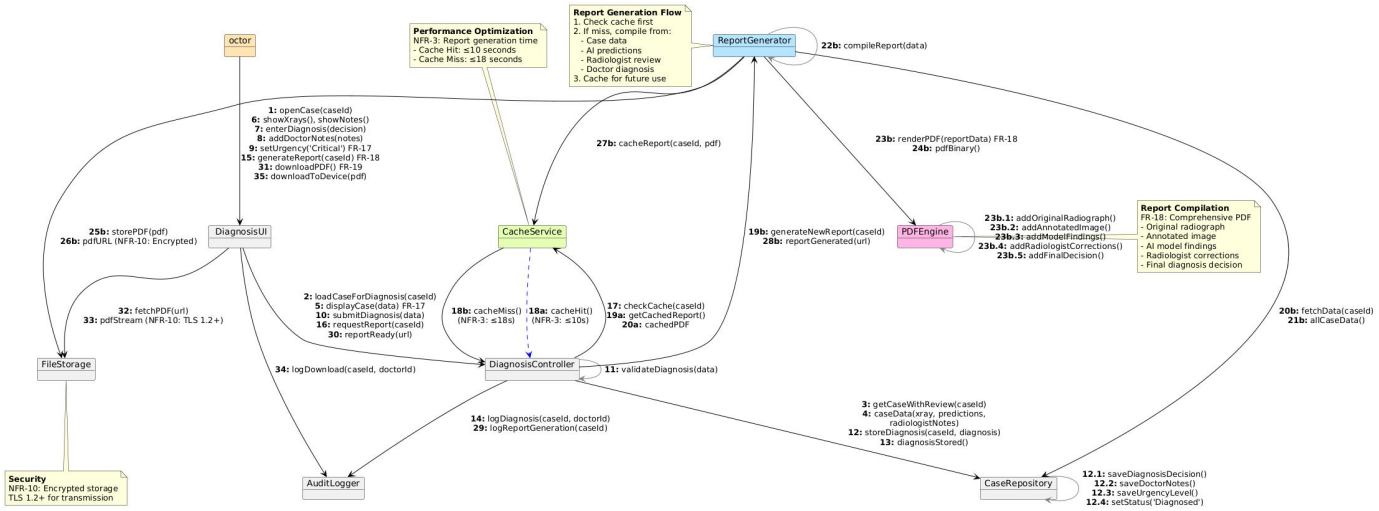

Image 18 Doctor Diagnosis and Report Generation Collaboration

3.3.5.4.4 Admin User Management Collaboration

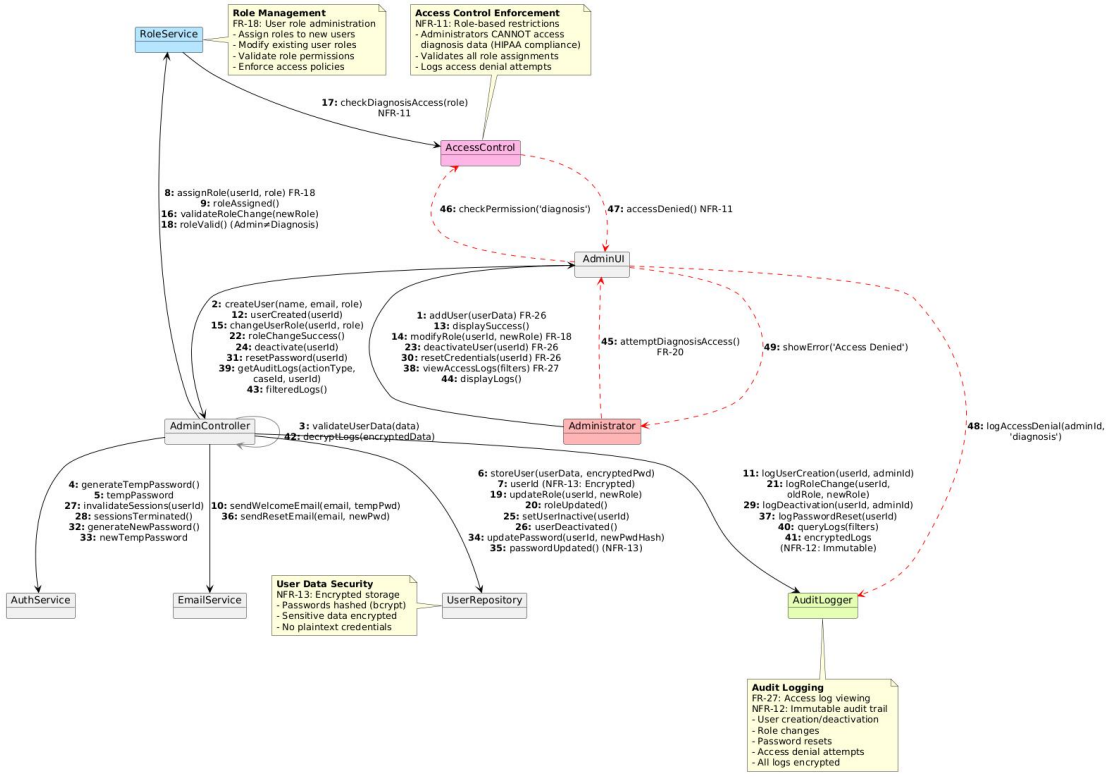

Image 19 Admin User Management Collaboration

3.6. Model Validation

Model validation was conducted to ensure that all system models developed in the analysis phase accurately and comprehensively represent the requirements of the AI-

Powered Lung Disease Detection and Clinical Decision Support System. The validation process sought to confirm that the models are correct, complete, consistent, and aligned with stakeholder expectations. The activities performed included requirements-to-model verification, stakeholder-based validation, and internal consistency and completeness checking across all UML diagrams.

3.6.1. Requirements Validation

The first stage of validation focused on verifying the correspondence between the documented functional and non-functional requirements and their representation within the system models. Each requirement was traced to at least one UML artifact to ensure that the intended system behavior is accurately captured.

Functional requirements related to user authentication, account management, and access control are reflected in the use case diagrams, account state diagrams, and authentication sequence models. Image acquisition workflows are clearly represented within the corresponding activity, sequence, and state machine diagrams.

The core system functionalities involving AI inference, bounding-box–based detection, and confidence scoring are captured in the Image Upload and Processing sequence diagrams as well as the model interaction workflows. Requirements for radiologist review, annotation, modification of detections, and doctor diagnosis finalization are accurately modeled in the Radiologist Review and Doctor Report Generation diagrams. Additionally, system behaviors tied to report generation, case storage, audit logging, and administrative management appear appropriately across the activity and state machine diagrams.

Furthermore, key non-functional requirements are reflected in the operational flows, such as encrypted transmission, data access checks, automatic retry logic, and backup operations.

Through this validation process, it was confirmed that each functional and non-functional requirement is represented within at least one system model and that no diagram introduces behavior outside the defined project scope

3.6.2. Stakeholder Validation

The second stage of validation involved reviewing the system models against the expectations and workflows of the key stakeholders identified in the project: laboratory technicians (radiologists), doctors and system administrators.

Stakeholder-oriented examinations confirmed that the diagrams accurately reflect the real-world clinical workflow. The radiologist-centric models correctly illustrate annotation, correction of AI detections, and confirmation stages. Similarly, the doctor-oriented diagrams accurately capture diagnosis review and final report approval processes. Administrative use cases , like managing user accounts, viewing audit logs, and assigning roles, are also coherently represented.

Stakeholder feedback incorporated into the modeling process resulted in refinements such as improved representations of annotation capabilities, expanded image format support, and stricter access-control mechanisms. These enhancements ensure that the system models reflect realistic clinical needs, thereby increasing stakeholder confidence in the accuracy and practicality of the modeled workflows.

3.6.3. Consistency and Completeness Checking

The final stage of validation involved examining internal consistency across all UML models and verifying that the diagrams collectively represent the complete scope of the system.

Consistency checks confirmed that:

- All actors appearing in activity, sequence, and state machine diagrams are also represented in the use case diagrams.
- State transitions for patient cases staring from upload to inference, review, finalization to archival and their behaviors illustrated in the dynamic diagrams are coherent.
- The relationships and attributes implied in the models correspond with those represented in the data structures, storage requirements, and system workflow descriptions.
- System behaviors such as authentication, validation, inference retry logic, and access control are represented uniformly across the diagrams.

Completeness checks demonstrated that all critical processes are represented with at least one dynamic model. UML correctness was also verified to ensure that diagram

semantics, actor interactions, and state transitions adhere to standard modeling conventions.

Minor gaps were identified, particularly in relation to the absence of a model lifecycle diagram for AI training and version management, and the lack of explicit diagrams for disaster recovery and performance testing scenarios. These gaps do not undermine the clarity or accuracy of the existing models but are recommended for inclusion to further strengthen system completeness.

Chapter Four: System Design

4.1. Overview

This chapter presents the comprehensive system design for the AI-powered lung disease detection infrastructure utilizing the YOLO (You Only Look Once) model. The design phase is crucial in translating the requirements specified in Chapter Three into a concrete, implementable architecture that addresses Ethiopia's unique healthcare challenges.

The system design encompasses several interconnected layers: a robust software architecture that supports scalability and maintainability, a decomposed subsystem structure that enables modular development and deployment, a secure and efficient database schema for persistent data management, a deployment architecture optimized for cloud-based nationwide access, and an intuitive user interface that prioritizes clinical usability and accessibility.

The design adheres to industry best practices including separation of concerns, defense-in-depth security (with TLS 1.2+ encryption for transmission and AES-256 for storage), comprehensive audit logging for regulatory compliance, and role-based access control preventing administrators from accessing sensitive diagnosis data. Furthermore, the architecture supports Ethiopia's multilingual requirements and ensures data minimization principles to protect patient privacy.

This chapter serves as the blueprint for implementation, guiding the development team through technical decisions while ensuring alignment with stakeholder needs, clinical workflows, and resource constraints inherent in Ethiopia's healthcare landscape.

4.2. Specifying the Design Goals

The design goals for the X-ray and CT scan analysis system are derived directly from the project's overarching objectives and must harmonize with both functional and non-functional requirements. These goals serve as guiding principles throughout the design and implementation phases.

Performance and Efficiency: The system must deliver rapid diagnostic support to clinicians. The primary performance goal is achieving end-to-end image analysis—from upload through preprocessing, AI inference, and result storage—within ≤12 seconds on a single GPU machine. Additionally, user interface interactions such as toggling annotation overlays or adjusting confidence filter thresholds must render within ≤300 milliseconds to ensure responsive clinical workflows. Report generation must complete within ≤10 seconds for cached reports and ≤18 seconds for newly generated reports. These aggressive timing constraints necessitate optimized model architectures (utilizing lightweight YOLOx variants), efficient preprocessing pipelines (leveraging GPU-accelerated operations), and intelligent caching strategies.

Scalability: The system must scale horizontally to accommodate Ethiopia's growing healthcare infrastructure. The design goal is to support at least 200 simultaneous logged-in users without performance degradation or failure, distributed across multiple hospitals and clinics nationwide. This requires load-balanced API servers, distributed session management using Redis, connection pooling for database access, and cloud-based deployment with auto-scaling capabilities. The architecture must also accommodate future growth in data volume as the system processes thousands of X-ray images monthly.

Reliability and Availability: Healthcare systems demand high reliability. The design goal is to achieve ≥95% system uptime, supported by automated health monitoring, automatic service restart on failure, and disaster recovery capabilities that restore from backups within 2 hours. Daily and weekly automated backups with encrypted archival (AES-256) ensure data durability. The system incorporates retry logic for transient failures, queuing failed inference jobs for administrative review, and maintains redundant infrastructure components to eliminate single points of failure.

Security and Privacy: Given the sensitivity of medical data, security is paramount. Design goals include end-to-end encryption (TLS 1.2+ for data in transit and AES-256 for data at rest) ensuring no plaintext credentials, diagnosis information, or patient data exist in databases or source code. Role-based access control enforces the principle of least privilege, particularly ensuring administrators cannot access diagnostic content. Comprehensive immutable audit logging tracks all critical actions including logins, image uploads, AI predictions, diagnosis submissions, and report

generation. Patient data minimization and explicit consent workflows align with ethical standards.

Usability and Accessibility: The system must be accessible to clinicians with varying technical proficiency. Design goals include a learning curve where users complete workflows (upload, review, diagnose) within 10 minutes of first usage , clear visual interfaces with zoom controls, annotation tools, and toggle views, contextual help documentation with tooltips and onboarding tutorials, and WCAG 2.1 AA compliance for accessibility including screen reader support. Multilingual support for English and Amharic with locale-based date/time formatting ensures cultural relevance.

Interoperability and Compatibility: To integrate with Ethiopia's evolving digital health ecosystem, the design supports RESTful API endpoints with JSON responses, compatibility with modern web browsers including Chrome, Firefox, Edge, and Safari, and standardized data formats (DICOM for medical imaging). This ensures the system can exchange data with external hospital information systems (HIS) and national health databases.

4.3. System Design

4.3.1. Proposed Software Architecture

The system adopts a layered architecture combined with microservices principles to achieve separation of concerns, scalability, and maintainability. This hybrid approach allows for modular development while maintaining clear boundaries between presentation, business logic, and data access layers.

Architectural Style: The architecture follows a three-tier pattern with additional specialized services:

1. Presentation Layer (Frontend): A React.js-based website that runs in the user's browser, providing an interactive and responsive user interface. This layer communicates exclusively with the backend through RESTful APIs over HTTPS, ensuring secure client-server communication. The frontend implements role-based view rendering, displaying different dashboards and functionalities for Lab Technicians/Radiologists, Doctors, and Administrators as defined by user roles.
2. Application Layer (Backend API): A FastAPI-based Python web framework serving as the central orchestrator for business logic and system workflows. This

layer exposes RESTful endpoints for authentication, image upload and validation, case management, diagnosis submission, report generation, and administrative functions. The backend implements middleware for authentication token validation, role-based authorization, request logging, and error handling with automatic retry logic.

3. AI Inference Service: A specialized microservice encapsulating the YOLOx deep learning model for pneumonia, tuberculosis, and lung tumor detection. This service is decoupled from the main backend to enable independent scaling, model updates without API downtime, and GPU resource management. The service receives preprocessed images, executes inference generating bounding boxes and confidence scores, and returns structured prediction results within the ≤12-second performance constraint. The design supports model versioning, allowing A/B testing of improved models and rollback capabilities.

4. Data Layer: Comprises a PostgreSQL relational database for structured data (user accounts, patient records, cases, diagnoses, audit logs) and object storage (minio) for unstructured data (raw X-ray images, annotated images, PDF reports). PostgreSQL provides ACID compliance essential for medical data integrity, while object storage offers cost-effective scalability for large binary files with AES-256 encryption at rest.

5. Supporting Services:

    - Authentication Service: Manages user sessions, JWT token generation/validation, two-factor authentication via email OTP, and password hashing with bcrypt/argon2.
    - Image Preprocessing Service: Handles uploaded images, performs validation checks (file type, size, resolution, headers), duplicate detection using SHA-256 hashing, and applies preprocessing transformations
    - Report Generation Service: Compiles data from AI predictions, radiologist annotations, and doctor diagnoses to generate PDF medical reports. Implements caching mechanisms to meet the ≤10-second cached and ≤18-second non-cached generation requirements.
    - Audit Logger: A dedicated service recording immutable encrypted logs for all critical system actions (logins, uploads, predictions, diagnosis

modifications, deletions, report generation) to satisfy compliance and security audit requirements.

- Email Service: Sends OTP codes for two-factor authentication, password reset emails, and notification alerts to clinicians.

The communication Patterns used allows services communicate via synchronous HTTP/REST for user-facing operations requiring immediate responses and asynchronous message queues (implemented with Redis or RabbitMQ) for background tasks such as AI inference scheduling, backup operations, and email sending. This hybrid approach ensures low-latency user interactions while offloading time-intensive operations to prevent blocking.

A load balancer (Nginx) distributes incoming requests across multiple backend API server instances, enabling horizontal scaling to support 200+ concurrent users (NFR-4). Session state is externalized to a Redis cluster, allowing any backend instance to serve requests for any user, facilitating stateless server design and seamless failover.

Security Architecture: Defense-in-depth strategies are employed:

- Transport Security: All client-server and service-to-service communication occurs over TLS 1.2+.
- Authentication & Authorization: JWT tokens with short expiration times, role-based access control (RBAC) enforcing permissions at API endpoints (e.g., only doctors can finalize diagnoses), and input validation preventing injection attacks.
- Data Encryption: Sensitive fields (credentials, patient data, diagnoses) are encrypted with AES-256 in the database; file storage is encrypted at rest.
- Audit Logging: Immutable logs recorded for forensic analysis and regulatory compliance.

The system is containerized using Docker, with each service packaged as an independent container, simplifying deployment and ensuring consistency across development, testing, and production environments. Kubernetes or Docker Compose manages container orchestration, health checks, automatic restarts, and resource allocation. The architecture supports both cloud-based access for hospitals with reliable internet and future offline-capable edge deployments using lightweight models for remote clinics.

This proposed architecture balances performance, security, scalability, and maintainability, positioning the system to meet Ethiopia's diagnostic challenges while accommodating future enhancements such as integration with Electronic Health Records (EHR), mobile applications for rural health workers, and expansion to additional disease detection capabilities.

4.3.2. Subsystem Decomposition

The system is decomposed into eight major subsystems, each encapsulating specific functional responsibilities. This modular decomposition facilitates parallel development by independent teams, enables targeted testing and debugging, and supports incremental deployment and scaling.

1. Authentication & User Management Subsystem

Responsibilities: User registration, login authentication with two-factor verification, session management, role assignment and modification, password resets, and user account lifecycle (activation, deactivation, deletion).

Components:

- User Repository: Database access layer for CRUD operations on user accounts, storing encrypted credentials and roles.
- Authentication Controller: Validates login credentials, generates OTP codes, sends verification emails, creates JWT tokens upon successful authentication.
- Session Manager: Manages active user sessions, token validation, session expiration, and logout.
- Role Service: Enforces role-based permissions, ensuring Administrators cannot access diagnosis dat.

2. Image Upload & Validation Subsystem

Responsibilities: Accepting chest X-ray and CT scan uploads, validating file format, size, resolution, and headers, detecting duplicate images using hash comparison, and storing raw images securely.

Components:

- Upload Controller: Handles HTTP file uploads, orchestrates validation and storage workflows.

- Image Validator: Checks file type (PNG, JPEG, DICOM), size limits, pixel resolution constraints, and validates DICOM headers for integrity.
- Duplicate Detector: Computes SHA-256 hashes of uploaded images, queries the database for existing hashes, alerts users if duplicates exist, and allows override confirmations.
- File Storage Service: Stores validated images in cloud object storage with AES-256 encryption, returns file URLs for database persistence.

3. AI Inference & Preprocessing Subsystem

Responsibilities: Preprocessing uploaded images, executing AI-based disease detection using YOLOx, generating bounding boxes around abnormalities, calculating class probabilities and confidence scores, and storing inference results. The entire workflow must complete within ≤12 seconds.

Components:

- Image Preprocessor: Resizes images to the model's input dimensions, normalizes pixel values (0-1 range), applies Contrast Limited Adaptive Histogram Equalization (CLAHE) for dynamic contrast enhancement, optimizing image quality for AI analysis.
- YOLOx Inference Engine: Loads the trained YOLOx model weights, executes forward pass to detect abnormal regions, classifies regions into disease categories (Pneumonia, Tuberculosis, Lung Tumor), generates bounding box coordinates, and computes confidence scores for each detection.
- Result Repository: Stores raw image URLs, prediction objects (disease classes, bounding boxes, confidence scores), and inference metadata (model version, processing time) in the database.
- Interfaces: Internal API invoked by Upload Controller post-validation, asynchronous processing via message queue for non-blocking operation, retry logic queuing failures for admin review.

4. Radiologist Review & Annotation Subsystem

Responsibilities: Presenting AI predictions to radiologists, enabling interactive annotation editing (correcting bounding boxes, removing false positives, adding

missed detections, modifying disease classifications), adjusting confidence filter thresholds, and capturing radiologist notes.

Components:

- Case Repository: Fetches case data including patient information, X-ray images, and AI predictions.
- Annotation Editor: Provides tools for manipulating bounding boxes (resize, move, delete, add new), changing disease class labels, and real-time rendering of edits with <300ms interaction latency.
- Filter Engine: Applies confidence threshold filters, dynamically updating displayed annotations based on user-selected thresholds (0-100%).
- Image Renderer: Renders high-resolution X-ray images with overlayed bounding boxes, zoom/pan controls, and toggle views (original vs. annotated).

5. Diagnosis & Decision Subsystem

Responsibilities: Enabling doctors to review complete case data (patient history, X-rays, AI findings, radiologist notes), submit final diagnoses with urgency levels (Critical/Non-Critical), add doctor notes, and generate comprehensive medical reports.

Components:

- Diagnosis Controller: Orchestrates diagnosis workflow, validates inputs, stores diagnosis decisions encrypted (AES-256).
- Report Generator: Compiles data from multiple sources (original radiograph, annotated image, model findings, radiologist corrections, final diagnosis), formats content into structured PDF documents using library like ReportLab or WeasyPrint, implements caching for ≤10-second cached and ≤18-second non-cached generation.
- PDF Engine: Renders PDF files with embedded images, tables, and formatted text, supports digital signatures for doctor attestation.
- Case History Service: Retrieves patient historical cases for comparison and longitudinal analysis.

6. Patient & Case Management Subsystem

Responsibilities: Registering new patients, linking follow-up visits to existing patients, searching case histories by patient ID, diagnosis type, and date range, ensuring data minimization and consent recording.

Components:

- Patient Repository: CRUD operations for patient records storing minimal essential data (Patient ID, Age, Sex, Visit Date).
- Case Service: Creates new cases, links cases for follow-up visits, manages case statuses (Pending Review, Ready for Diagnosis, Diagnosed, Completed).
- Search Engine: Executes multi-criteria queries filtering cases by patient demographics, diagnosis types, date ranges, and urgency levels.
- Consent Manager: Records patient consent acknowledgments, timestamps, and reference numbers for ethical compliance .

7. Administrative & Monitoring Subsystem

Responsibilities: User management (adding, deactivating, resetting credentials, assigning roles as per), viewing system-wide access logs filtered by action type, case ID, user ID, monitoring system health, and enforcing the constraint that admins cannot approve diagnose.

Components:

- Admin Controller: Handles administrative operations, validates role changes preventing admins from accessing diagnosis content.
- Audit Log Viewer: Queries encrypted immutable audit logs, decrypts for authorized admin viewing, supports filtering and pagination.
- Monitoring Service: Tracks system metrics (uptime, response times, error rates), integrates with alerting tools for proactive issue detection.
- Backup Service: Orchestrates daily and weekly encrypted backups, stores incremental and full snapshots, supports disaster recovery restoring data within 2 hours.

4.3.3. Database Design

The database schema is designed to support efficient data storage, retrieval, and integrity while ensuring security and compliance with privacy regulations. PostgreSQL is chosen for its robustness, support for complex queries, transactional integrity, and extensibility.

The database comprises the following core entities, relationships, and constraints:

1. Hospitals
    - Purpose: Stores hospital information for multi-hospital deployment
    - Attributes
        - hospital_id (UUID, PK): Unique hospital identifier
        - hospital_name (VARCHAR, UK, NOT NULL): Unique hospital name
        - address (VARCHAR, NOT NULL): Hospital physical address
        - city (VARCHAR, NOT NULL): City where hospital is located
        - phone_number (VARCHAR, NOT NULL): Hospital contact number
        - email (VARCHAR, UK, NOT NULL): Unique hospital email for official communication
        - status (ENUM): `Active`, `Inactive` (Default: Active)
        - created_at (TIMESTAMP): Hospital registration timestamp
        - updated_at (TIMESTAMP): Last update timestamp
    - Implements: Multi-hospital support, allowing the system to serve multiple hospitals with isolated user management
    - Constraints:
        - hospital_name must be unique
        - email must be unique
        - Check constraint on status to enforce valid values
    - Relationships:
        - One-to-Many with `Users` (a hospital can have multiple users including multiple admins)

2. Users

- Purpose: Stores all system users (Lab Technicians, Radiologists, Doctors, Administrators)
- Attributes:
  - user_id (UUID, PK): Unique identifier
  - hospital_id (UUID, FK, NOT NULL): References Hospitals - Hospital where user is employed
  - email (VARCHAR, UK, NOT NULL): Unique email for login
  - password_hash (VARCHAR, NOT NULL): Encrypted password (NFR-14)
  - name (VARCHAR, NOT NULL): User's full name
  - role (ENUM, NOT NULL): One of Lab_Technician, Radiologist, Doctor, Admin
  - status (ENUM): Active, Inactive, Locked (Default: Active)
  - created_at (TIMESTAMP): Account creation timestamp
  - last_login (TIMESTAMP): Last successful login
  - phone_number (VARCHAR, NULLABLE): Optional phone number
- Implements: FR-01 (User Login), FR-02 (2FA), FR-03 (Role Management), FR-04 (Logout)
- Constraints:
  - Check constraint on `role` to enforce valid values (FR-03)
  - No plaintext passwords stored (NFR-14)
  - email must be unique
  - Multiple admins can exist per hospital
- Relationships:
  - Many-to-One with Hospitals (user belongs to one hospital)
  - One-to-Many with Cases (as upload technician)

- One-to-Many with Radiologist_Reviews (as radiologist)
- One-to-Many with Diagnoses (as diagnosing doctor)
- One-to-Many with Reports (as report generator)
- One-to-Many with Audit_Logs (as actor)
- One-to-Many with Sessions

3. Patients
   - Purpose: Stores minimal patient demographic information
   - Attributes:
     - patient_id (UUID, PK): Unique patient identifier
     - age (INTEGER, NOT NULL): Patient age
     - sex (ENUM, NOT NULL): Male, Female
     - registered_at (TIMESTAMP): Registration timestamp
     - consent_recorded (BOOLEAN): Default: false (NFR-26)
     - consent_date (TIMESTAMP, NULLABLE): Date consent was provided
   - Implements: FR-21 (Register New Patient Record), NFR-25 (Data Minimization)
   - Constraints:
     - Minimal data storage per NFR-25
     - No unnecessary fields like full name or address unless essential
   - Relationships:
     - One-to-Many with Cases.
4. Cases
   - Purpose: Represents individual medical cases/visits
   - Attributes:
     - case_id (UUID, PK): Unique case identifier
     - patient_id (UUID, FK, NOT NULL): References Patients

- upload_tech_id (UUID, FK, NULLABLE): References Users (Lab Technician who uploaded)
- visit_date (DATE, NOT NULL): Date of visit
- status (ENUM): Pending_Review, In_Review, Ready_for_Diagnosis, Diagnosed, Completed
- created_at (TIMESTAMP): Case creation timestamp
- updated_at (TIMESTAMP): Last update timestamp
- linked_case_id (UUID, FK, NULLABLE): Self-reference for follow-up cases (FR-24)
- priority (ENUM, NULLABLE): Critical, Non_Critical

- Implements: FR-24 (Case Linking), FR-25 (Patient-Based Search)
- Constraints:
  - Foreign key constraints ensuring referential integrity
  - Default status: Pending_Review
- Relationships:
  - Many-to-One with Patients
  - Many-to-One with Users (upload tech)
  - One-to-Many with Images
  - One-to-One with Radiologist_Reviews
  - One-to-One with Diagnoses
  - One-to-Many with Reports
  - One-to-Many with Audit_Logs
  - Self-referencing for follow-up cases

5. Images

- Purpose: Stores uploaded X-ray image metadata
- Attributes:
  - image_id (UUID, PK): Unique image identifier

- case_id (UUID, FK, NOT NULL): References Cases
- file_url (VARCHAR, NOT NULL): URL to encrypted cloud storage
- file_hash (VARCHAR(64), UK): SHA-256 hash for duplicate detection (FR-07)
- file_format (ENUM): PNG, JPEG, DICOM
- file_size_bytes (INTEGER): File size in bytes
- uploaded_at (TIMESTAMP): Upload timestamp
- metadata (JSONB): DICOM tags or additional information

- Implements: FR-05 (Upload Chest X-ray), FR-06 (Validation), FR-07 (Duplicate Detection)
- Constraints:
  - Unique constraint on file_hash supports duplicate detection
  - File content encrypted at rest via cloud storage (NFR-11)
- Relationships:
  - Many-to-One with Cases
  - One-to-Many with Inference_Results

6. Inference_Results

- Purpose: Stores AI model predictions
- Attributes:
  - inference_id (UUID, PK): Unique inference identifier
  - image_id (UUID, FK, NOT NULL): References Images
  - model_version (VARCHAR, NOT NULL): Version of AI model used (e.g., "YOLOx-v1.2.3")
  - processing_time_sec (FLOAT): Processing time in seconds (NFR-1)
  - predictions (JSONB): Array storing bounding boxes, classes, confidence scores (FR-10, FR-11)
  - created_at (TIMESTAMP): Inference completion timestamp

- Implements: FR-08 (Preprocessing), FR-09 (AI Detection), FR-10 (Bounding Boxes), FR-11 (Confidence Scores), FR-12 (Result Storage)
- Constraints:
  - JSONB type for flexible storage of variable-length prediction arrays
  - Performance target: ≤12 seconds end-to-end (NFR-1)
- Relationships:
  - Many-to-One with Images

7. Radiologist_Reviews

- Purpose: Stores radiologist corrections and annotations
- Attributes:
  - review_id (UUID, PK): Unique review identifier
  - case_id (UUID, FK, UK, NOT NULL): References Cases (unique - one review per case)
  - radiologist_id (UUID, FK, NOT NULL): References Users (must be Radiologist role)
  - reviewed_at (TIMESTAMP): Review completion timestamp
  - annotations (JSONB): Edited bounding boxes, added/removed detections (FR-15)
  - notes (TEXT, NULLABLE): Radiologist notes
  - confidence_threshold_applied (INTEGER 0-100, NULLABLE): Threshold used (FR-16)
- Implements: FR-13 (Review Predictions), FR-14 (Edit Annotations), FR-15 (Adjust Filters)
- Constraints:
  - One-to-one relationship with Cases ensured by unique constraint on case_id
  - Role check ensuring `radiologist_id` references a user with role `Radiologist
- Relationships:

- One-to-One with Cases
- Many-to-One with Users (radiologist)

8. Diagnoses

- Purpose: Stores final doctor diagnoses
- Attributes:
  - diagnosis_id (UUID, PK): Unique diagnosis identifier
  - case_id (UUID, FK, UK, NOT NULL): References Cases (unique - one diagnosis per case)
  - doctor_id (UUID, FK, NOT NULL): References Users (must be Doctor role)
  - primary_diagnosis (ENUM, NOT NULL): Normal, Pneumonia, Tuberculosis, Lung_Tumor, Other
  - diagnosis_notes (TEXT, NOT NULL, ENCRYPTED): Encrypted diagnosis notes (AES-256, NFR-11, NFR-14)
  - urgency_level (ENUM, NOT NULL): Critical, Non_Critical (FR-17)
  - treatment_recommendations (TEXT, NULLABLE, ENCRYPTED): Encrypted treatment plan
  - diagnosed_at (TIMESTAMP): Diagnosis submission timestamp
- Implements: FR-17 (Final Diagnosis Submission), NFR-12 (Access Restrictions), FR-19 (Diagnosis Access)
- Constraints:
  - One-to-one with Cases
  - Role check ensuring doctor_id references a Doctor
  - Encryption of diagnosis_notes and treatment_recommendations (AES-256)
  - Admin cannot access this table (NFR-12, FR-19)
- Relationships:
  - One-to-One with Cases
  - Many-to-One with Users (doctor)

9. Reports

- Purpose: Stores generated medical reports
- Attributes:
  - report_id (UUID, PK): Unique report identifier
  - case_id (UUID, FK, NOT NULL): References Cases
  - pdf_url (VARCHAR, NOT NULL): URL to encrypted PDF in cloud storage
  - generated_at (TIMESTAMP): Report generation timestamp
  - generated_by (UUID, FK, NOT NULL): References Users (Doctor)
  - file_size_bytes (INTEGER): PDF file size
  - cached (BOOLEAN): Indicates if report was cached (performance tracking per NFR-3)
- Implements: FR-20 (Download Report), FR-21 (Generate Report), FR-22 (View Past Reports)
- Constraints:
  - PDF files encrypted at rest (NFR-11)
  - Performance target: ≤10 seconds if cached, ≤18 seconds if not (NFR-3)
- Relationships:
  - Many-to-One with Cases
  - Many-to-One with Users (generator)

10. Audit_Logs

- Purpose: Immutable audit trail of system actions
- Attributes:
  - log_id (UUID, PK): Unique log identifier
  - user_id (UUID, FK, NULLABLE): References Users (nullable for system events)
  - action_type (ENUM, NOT NULL): Action performed
  - case_id (UUID, FK, NULLABLE): References Cases if applicable

- timestamp (TIMESTAMP, NOT NULL): Action timestamp
- ip_address (VARCHAR, NULLABLE): IP address of requester
- details (JSONB, ENCRYPTED): Encrypted action-specific data (e.g., modified fields)

- Implements: FR-27 (Access Log Viewing), NFR-13 (Audit Trail)
- Constraints:
  - Immutable logs (insert-only table, no updates/deletes) to ensure integrity (NFR-13)
  - Encrypted details field (AES-256)
  - Admin-only read access (FR-27)
- Relationships:
  - Many-to-One with Users
  - Many-to-One with Cases

11. Sessions

- Purpose: Manages user authentication sessions
- Attributes:
  - session_id (UUID, PK): Unique session identifier
  - user_id (UUID, FK, NOT NULL): References Users
  - token_hash (VARCHAR): Hashed JWT token for revocation checks
  - created_at (TIMESTAMP): Session creation timestamp
  - expires_at (TIMESTAMP): Session expiration timestamp
  - last_accessed (TIMESTAMP): Last activity timestamp
- Implements: FR-01 (User Login), FR-04 (Logout), NFR-4 (Concurrent Sessions)
- Constraints:
  - Index on expires_at for efficient session cleanup
  - Support for 200+ concurrent sessions (NFR-4)
- Relationships:

- Many-to-One with `Users`

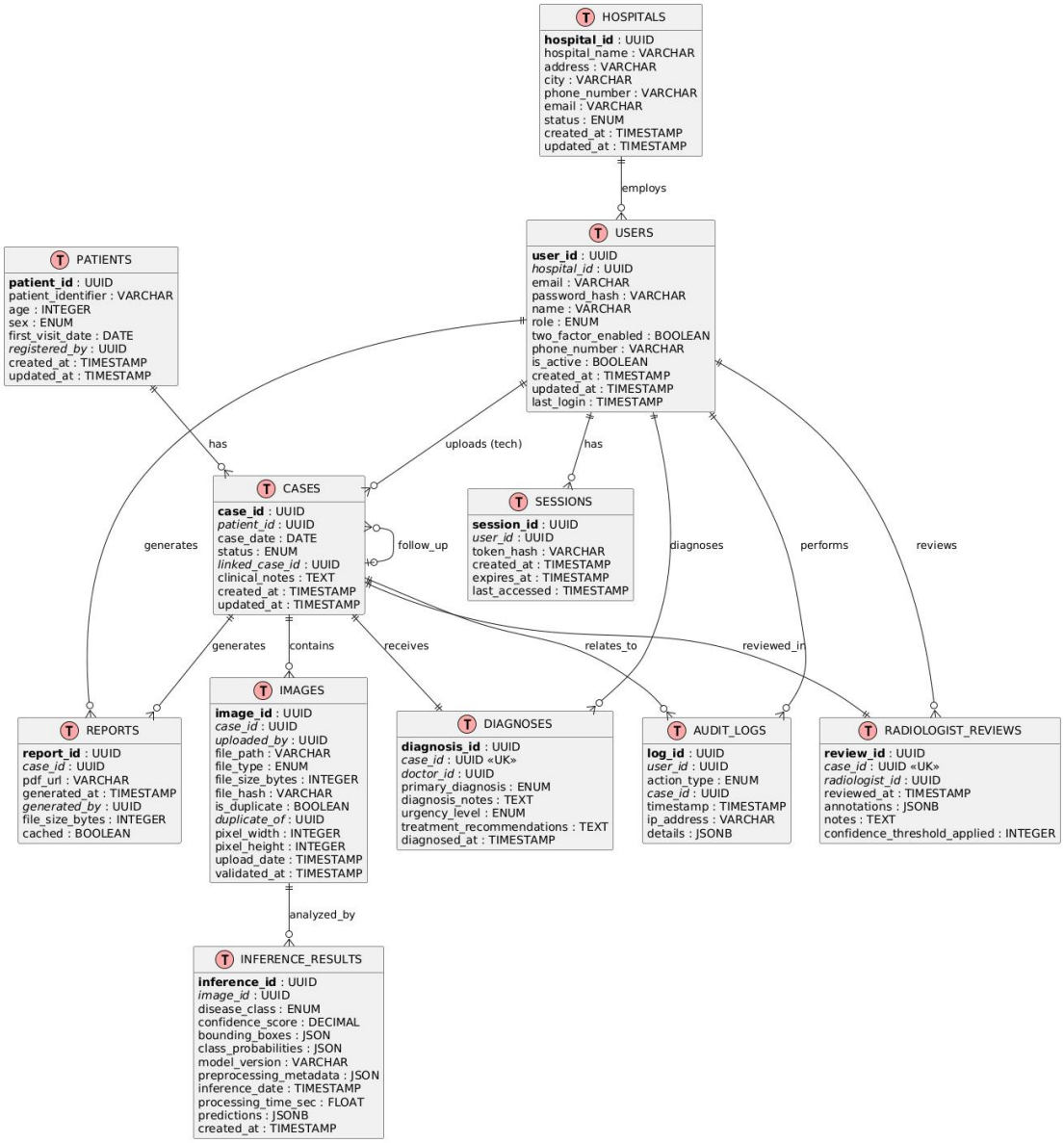

Image 20 Database Design Diagram

4.3.4. Deployment Diagram

The deployment diagram illustrates the physical architecture of the system, depicting how software components are distributed across hardware and cloud infrastructure nodes. This configuration is optimized for scalability, reliability, and security while accommodating Ethiopia's infrastructure realities.

Deployment Architecture Components:

1. Client Tier (User Devices):

- Components: Web browsers (Chrome, Firefox, Edge, Safari per NFR-21) running on desktop computers, laptops, and tablets used by Lab Technicians, Radiologists, Doctors, and Administrators.
- Communication: HTTPS connections over TLS 1.2+ (NFR-10) to the Load Balancer.
- Deployment: No installation required; users access the React.js SPA via standard web browsers, ensuring ease of access across hospitals and clinics.

2. Load Balancer / API Gateway Node:
   - Component: Nginx.
   - Responsibilities: Distributes incoming HTTPS requests across multiple Backend API Server instances for horizontal scaling (NFR-4), performs SSL termination (decrypting TLS traffic, re-encrypting for backend communication), implements rate limiting to mitigate denial-of-service attacks, and routes traffic based on URL paths (e.g., /api/auth/* to authentication services, /api/images/* to upload services).
   - Deployment: Deployed in a high-availability configuration with redundant instances across multiple availability zones to eliminate single point of failure.
3. Backend API Server Nodes (FastAPI Application):
   - Components: Multiple containerized FastAPI application instances (e.g., 3-5 instances initially, auto-scaled based on load).
   - Responsibilities: Serve RESTful API endpoints, execute business logic for authentication, case management, diagnosis submission, report generation, administrative functions, interact with Database Cluster, AI Inference Service, and File Storage.
   - Deployment: Deployed as Docker containers orchestrated by Kubernetes or Docker Swarm, enabling automatic scaling based on CPU/memory utilization or request queue length. Stateless design (session state externalized to Redis) allows seamless scaling and failover.

- Health Checks: Kubernetes livenessProbe and readinessProbe ensure unhealthy instances are restarted automatically.

4. Database Cluster Node (PostgreSQL):
   - Components: PostgreSQL database with primary-replica replication for read scalability and failover.
   - Responsibilities: Store and manage Users, Patients, Cases, Images metadata, Inference_Results, Radiologist_Reviews, Diagnoses, Reports, Audit_Logs, and Sessions.
   - Deployment: Managed database service (e.g., AWS RDS for PostgreSQL, Azure Database for PostgreSQL) configured with:
     - Primary Instance: Handles all write operations and consistent reads.
     - Read Replicas: Distribute read-heavy queries (e.g., case searches, report history) to reduce load on primary.
     - Automated Backups: Daily incremental and weekly full backups with point-in-time recovery.
     - Encryption: TLS for connections, AES-256 for data at rest.
   - High Availability: Multi-AZ deployment ensures automatic failover to standby replica if primary fails.
5. Redis Cache & Session Store Node:
   - Components: Redis cluster for session management and caching.
   - Responsibilities: Store user session tokens enabling stateless Backend API Servers, cache frequently accessed data (case details, report metadata) to reduce database load and meet <300ms UI interaction latency, cache generated reports for ≤10-second retrieval.
   - Deployment: Managed Redis service (e.g., AWS ElastiCache, Azure Cache for Redis) with persistence enabled for session durability.
6. Cloud Object Storage Node:
   - Components: AWS S3, Azure Blob Storage, or Google Cloud Storage buckets.

- Responsibilities: Store raw X-ray images, annotated images, and PDF reports with AES-256 encryption at rest
- Deployment: Configured with:
  - Private Access: Objects not publicly accessible; pre-signed URLs with expiration generated by Backend API for secure downloads.
  - Lifecycle Policies: Archive old images to lower-cost storage tiers (e.g., S3 Glacier) after 1 year to reduce costs.
  - Versioning: Enable versioning to prevent accidental deletions and support audit trails.

7. Email Service / SMTP Gateway:
   - Components: Third-party email service provider (e.g., SendGrid, Amazon SES, Mailgun) or self-hosted SMTP server.
   - Responsibilities: Send OTP codes for two-factor authentication, password reset emails, and notification alerts to doctors and radiologists.
   - Deployment: External SaaS service accessed via API over HTTPS; fallback SMTP server configured for redundancy.
8. Monitoring & Logging Node:
   - Components: Prometheus for metrics collection, Grafana for visualization, ELK Stack (Elasticsearch, Logstash, Kibana) or cloud-native solutions (AWS CloudWatch, Azure Monitor) for log aggregation.
   - Responsibilities: Monitor system health (CPU, memory, disk, network utilization), track application metrics (request latencies, error rates, inference times), aggregate logs from all services for centralized analysis, alert operations team on critical failures (triggered via email/SMS/Slack).
   - Deployment: Dedicated monitoring instances; Prometheus scrapes metrics endpoints from Backend API, AI Inference, and Database nodes; logs forwarded via log shippers (Filebeat) to Elasticsearch for indexing and querying.
9. Backup & Disaster Recovery Storage:

Components: Encrypted backup archives stored in geographically distributed cloud storage regions.

- Responsibilities: Store daily and weekly database backups , enable disaster recovery restoring full system state within 2 hours.
- Deployment: Separate cloud storage buckets with cross-region replication; automated backup scripts executed via cron jobs; retention policies delete backups older than 6 months to manage storage costs.

Network Configuration:

- Virtual Private Cloud (VPC): All backend components (API Servers, AI Inference, Database, Redis) deployed within private subnets of a VPC, isolating them from direct internet access.
- Public Subnets: Only the Load Balancer resides in public subnets, accepting incoming traffic.
- Security Groups/Firewall Rules: Restrict inbound traffic—only HTTPS (443) to Load Balancer, SSH (22) from bastion host for administration, internal communication between services on necessary ports.
- VPN/Bastion Host: Secure administrative access to backend infrastructure via VPN or bastion host, preventing direct SSH exposure.

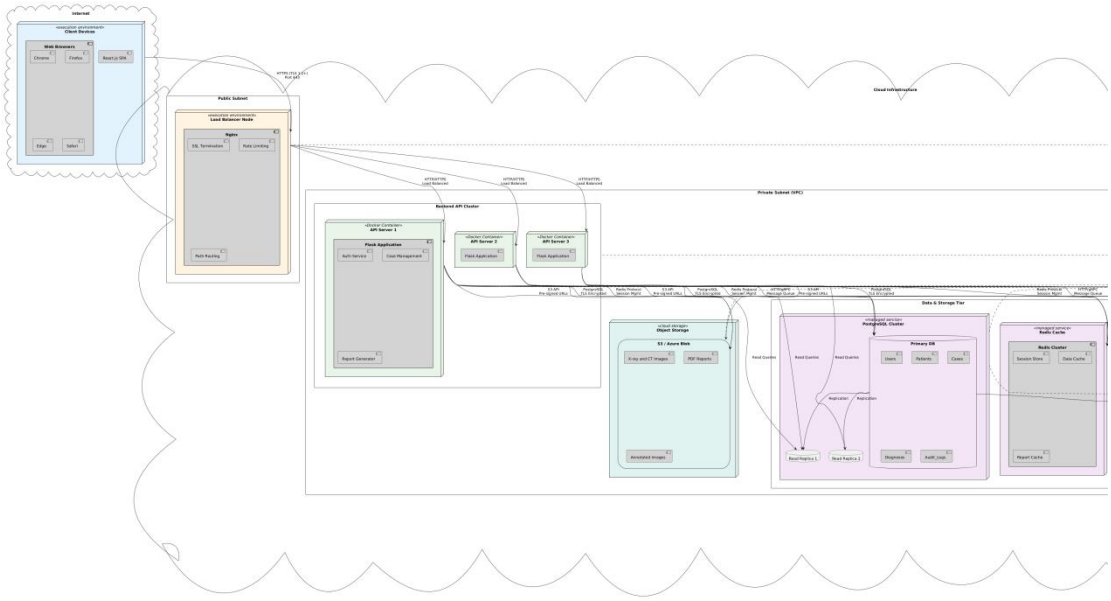

Image 21 Deployment Diagram

Failover and Redundancy: Load Balancer health checks detect unresponsive Backend API instances and route traffic to healthy instances. PostgreSQL primary-replica setup fails over to replica if primary becomes unavailable. Redis persistence ensures session data is not lost. Multi-region deployment (future enhancement) replicates critical components across geographic regions for disaster recovery beyond data backups.

4.3.5. User Interface Design

The user interface (UI) design prioritizes clinical usability, accessibility, and efficiency, ensuring healthcare professionals can seamlessly navigate workflows from image upload through diagnosis and report generation. The design adheres to established HCI principles, incorporates user feedback from formative evaluations (if conducted), and complies with accessibility standards.

Design Principles:

1. Simplicity and Clarity: Minimize cognitive load by presenting only essential information relevant to the user's current task. Use clear labeling, intuitive iconography, and consistent terminology aligned with medical practice.
2. Role-Based Interfaces: Tailor dashboards and navigation menus to user roles (Lab Technician/Radiologist, Doctor, Administrator), hiding irrelevant functionalities and emphasizing role-specific actions to reduce clutter and improve focus.
3. Visual Hierarchy: Employ typography, color, and spacing to establish clear visual hierarchies, guiding users' attention to primary actions (e.g., "Submit Diagnosis" button prominently styled) and critical information (e.g., red badges for critical cases).
4. Responsive Design: Ensure interfaces adapt gracefully to various screen sizes (desktop, laptop, tablet) commonly used in Ethiopian hospitals, though mobile-first is not prioritized given the clinical setting.
5. Accessibility (WCAG 2.1 AA Compliance): Implement sufficient color contrast (≥4.5:1 for normal text), keyboard navigation support, ARIA labels for screen readers, and alternative text for images and icons to assist users with visual impairments.

6. Multilingual Support: Provide language toggle between English and Amharic with culturally appropriate icons and symbols. Use Unicode fonts supporting Amharic script; ensure right-to-left text rendering if needed.

7. Feedback and Guidance: Offer real-time validation feedback (e.g., "Invalid file format" during upload), contextual help tooltips, progress indicators (upload progress, AI processing status), and confirmation dialogs for destructive actions (e.g., deleting annotations).

Key UI Pages and Elements:

1. Login Page:

- Layout: Centered card on a clean background featuring application logo, title, email and password input fields with validation indicators, "Forgot Password?" link, and primary "Sign In" button.
- Interaction: On submit, display loading spinner, upon successful credentials validation, navigate to 2FA page; on error, show inline error messages without clearing fields.

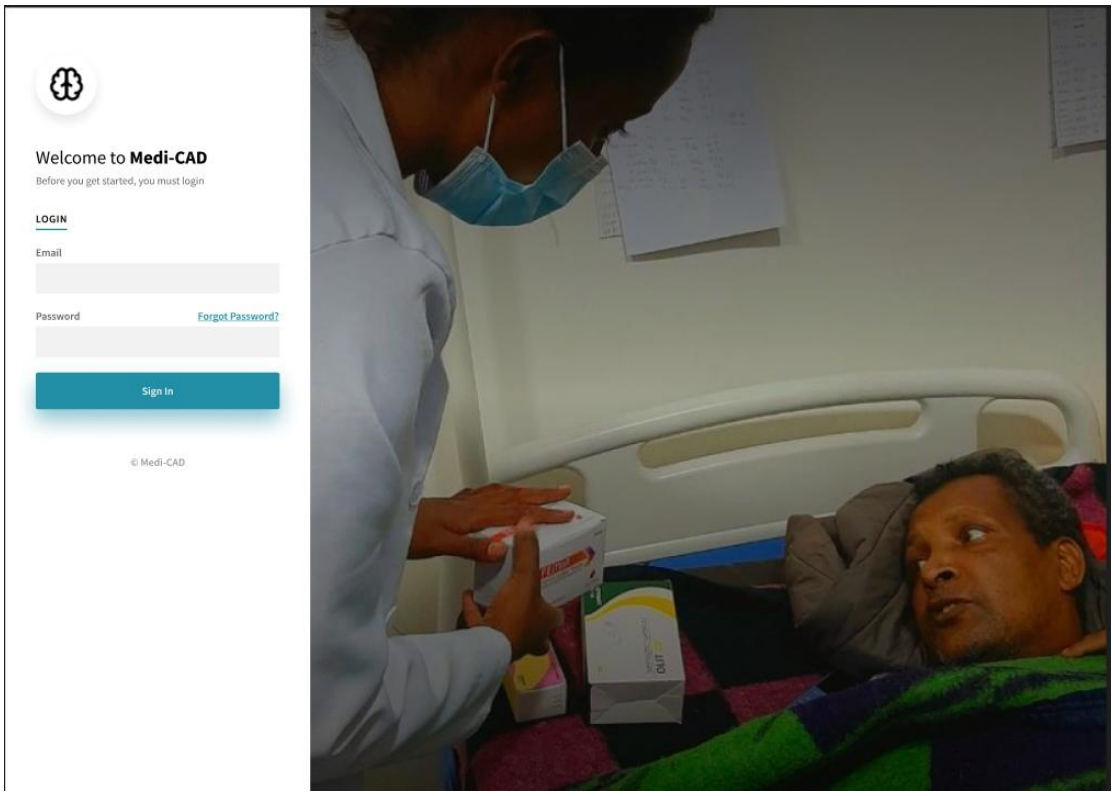

Image 22 Login UI

2. Two-Factor Authentication Page (FR-02):

- Layout: 6-digit OTP input field (either individual boxes for each digit or single field with masking), countdown timer showing OTP expiration (5 minutes), "Resend Code" button (disabled during countdown), and "Verify" button.
- Interaction: Auto-focus on OTP field; auto-submit upon entering 6 digits; real-time feedback on OTP validity; navigate to role-based dashboard upon successful verification.

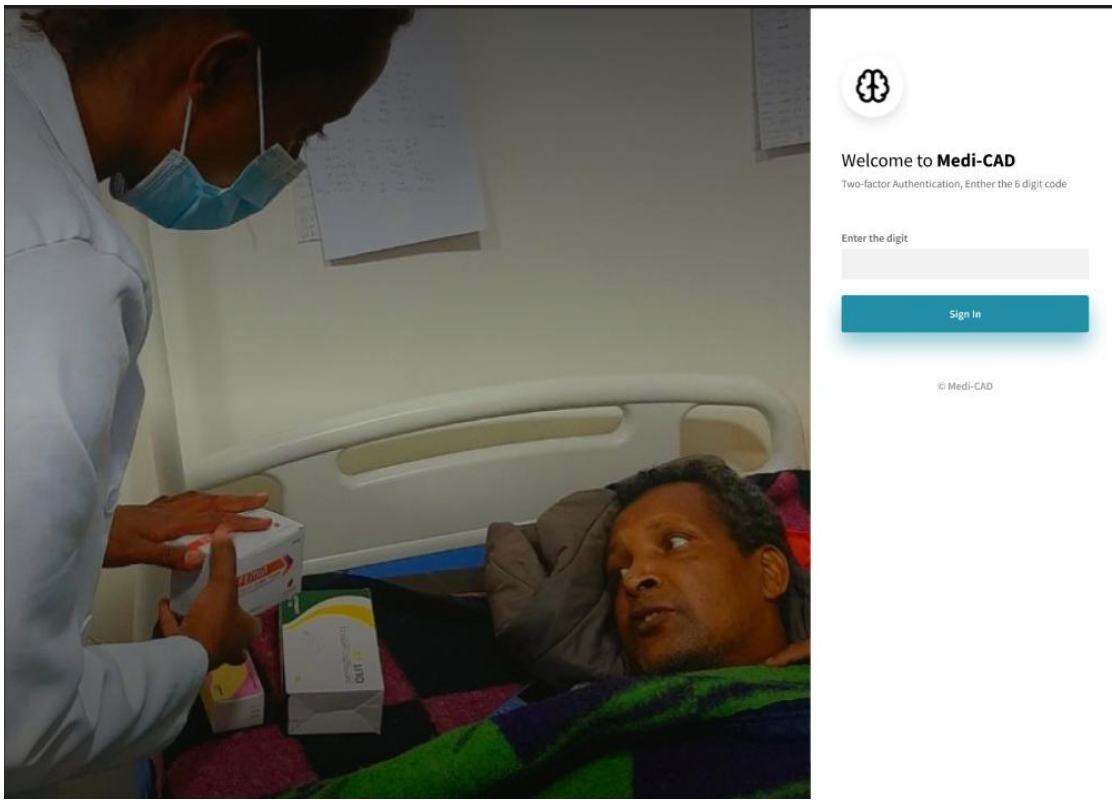

Image 23 Two-Factor Authentication UI

3. Lab Technician/Radiologist Dashboard:

- Components: Welcome message, quick stats cards (Pending Review count, Completed Today count), recent cases table with "Review" action buttons, and "Upload New X-ray" prominent button.
- Navigation: Side menu with icons: Dashboard (home), Upload, Cases Queue, Profile, Logout.

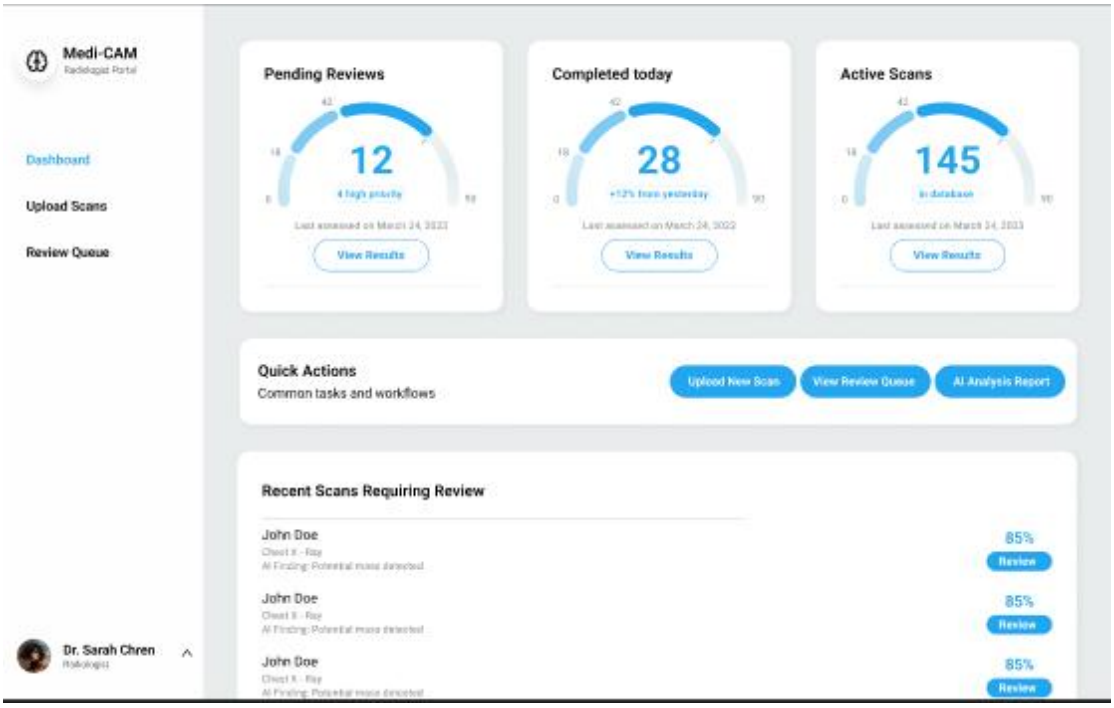

Image 24 Lab Technician/Radiologist Dashboard UI

4. Upload X-ray Image Page:

Layout: Two-column layout—left column for Patient Information (collapsible forms for New Patient and Existing Patient Search), right column for Image Upload section.

Upload Zone: Large drag-and-drop area with dashed border, "Drag & Drop X-ray images here or click to browse" text, accepted formats listed below, visual feedback on drag-over (highlight border).

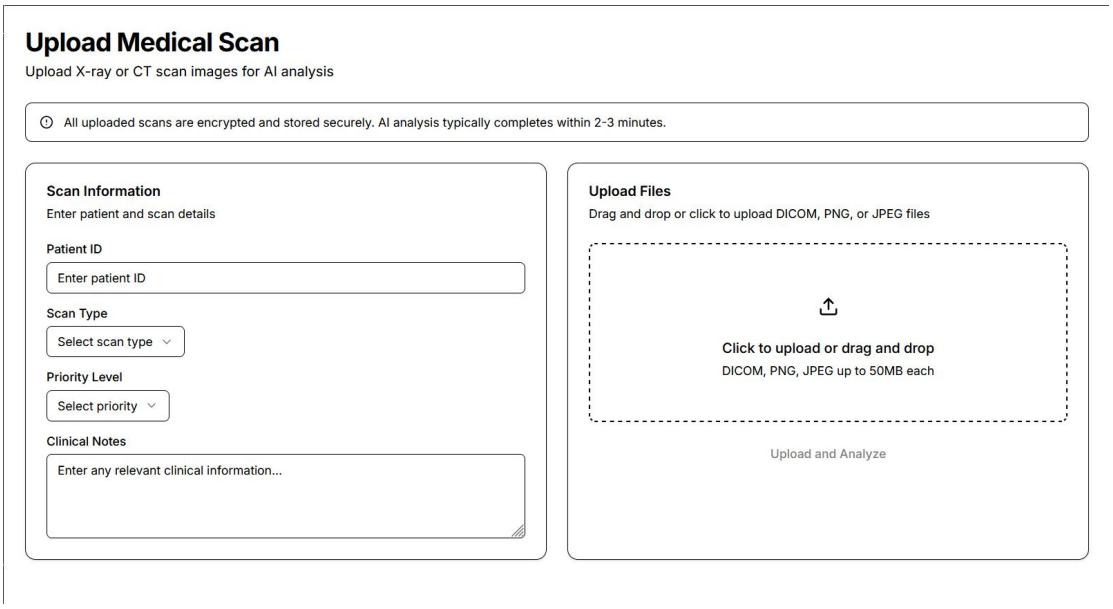

Image 25 Upload X-ray Image UI

5. Review AI Predictions Page:

- Layout: Split-screen—left 60% for Image Viewer, right 40% for Predictions Panel and Annotation Tools.
- Image Viewer: High-resolution X-ray display with overlayed bounding boxes color-coded by disease class (Pneumonia: red, TB: yellow, Tumor: orange). Interactive zoom/pan controls (zoom in/out buttons, mouse wheel zoom, click-and-drag pan). Toggle buttons: "Show/Hide Annotations," "Original/Annotated Image," "Show/Hide Scores."
- Predictions Panel: List of detected abnormalities—each item shows disease class icon, label, confidence score as progress bar and percentage. "Focus" button (centers image on detection), and "Edit"/"Remove" buttons .

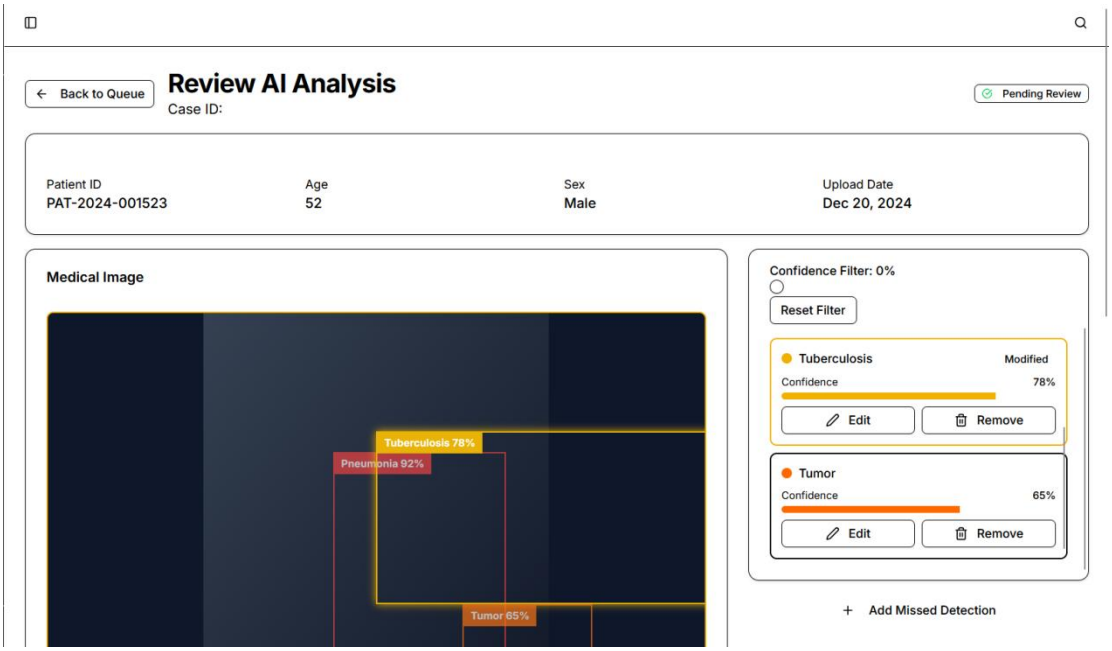

Image 26 Review AI Predictions UI

- Confidence Filter: Slider (0-100%) with live update filtering displayed annotations, current threshold value display, and "Apply" button (FR-16).
- Annotation Toolbar: Icons for Select/Move tool, Rectangle tool (draw new bounding box), Eraser, Text annotation. When editing a detection, panel displays disease class dropdown, resizable bounding box with corner/edge handles, notes textarea, and "Save Changes"/"Cancel" buttons.

- Radiologist Notes: Rich text editor at bottom of right panel with formatting tools (bold, italic, lists), auto-save indicator.ta.

6. Doctor Review & Diagnosis Page (FR-17):

- Layout: Three-column layout—left column: Patient Info card, Medical History timeline, Case History link; center column: Side-by-side X-ray image comparison (original vs. annotated) with zoom controls; right column: AI & Radiologist Analysis Panel and Diagnosis Form.
- Analysis Panel: Accordion sections: "AI Model Findings" (list of detections with confidence scores), "Radiologist Review" (radiologist notes, modifications summary, radiologist name/timestamp).
- Diagnosis Form: Dropdown/autocomplete for Primary Diagnosis (Normal, Pneumonia, TB, Lung Tumor, Other), optional Secondary Diagnosis, detailed notes rich text editor, Urgency Level radio buttons (Citical, Non-Critical with color indicators), Treatment Recommendations textarea, Doctor Notes textarea.
- Validation: Required field indicators (red asterisks), disabled "Submit Diagnosis" button until required fields complete, validation warnings if urgency not set for critical findings.
- Action Buttons: "Save as Draft," "Submit Diagnosis" (primary CTA, triggers confirmation modal summarizing diagnosis), "Generate Report", "Cancel.".

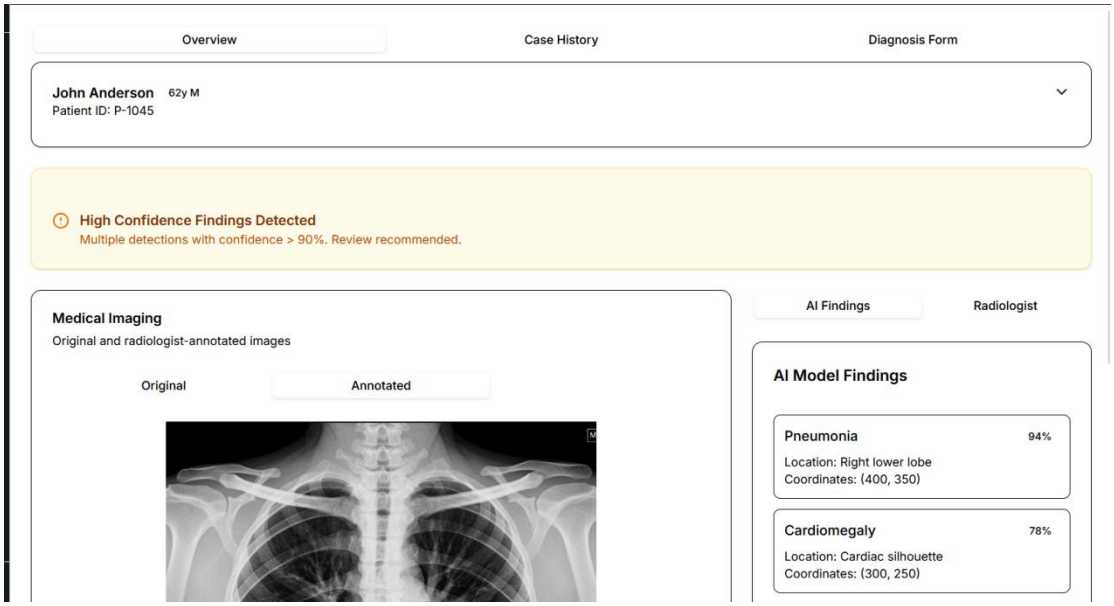

Image 27 Doctor Review & Diagnosis UI

4.3.6. Security Design

Security is a foundational pillar of the X-ray analysis system, given the highly sensitive nature of medical data and the critical importance of diagnostic integrity. This section presents a comprehensive security design addressing authentication, authorization, encryption, vulnerability mitigation, and compliance with healthcare data protection standards. The security architecture implements defense-in-depth principles, ensuring multiple layers of protection guard against diverse threat vectors.

4.3.6.1. Authentication Mechanisms

Multi-Factor Authentication:

The system implements a two-factor authentication (2FA) flow to verify user identity:

1. Primary Authentication:

- Users provide email and password credentials.
- Passwords must meet complexity requirements: minimum 12 characters, including uppercase, lowercase, digits, and special characters.
- Password hashing uses Argon2id or bcrypt (work factor ≥12) before storage in the Users table, ensuring irreversibility even if the database is compromised.
- Failed login attempts trigger exponential backoff (3 attempts → 1-minute lockout, 5 attempts → 15-minute lockout, 10 attempts → account locked pending admin review) to thwart brute-force attacks.

2. Secondary Authentication:

- Upon successful password verification, the system generates a 6-digit time-based one-time password (TOTP) valid for 5 minutes.
- OTP is sent to the user's registered email address via the Email Service (SendGrid/AWS SES) over HTTPS.
- OTP values are hashed (SHA-256) and stored temporarily in Redis with TTL (time-to-live) of 5 minutes, preventing replay attacks.
- Users enter OTP on the 2FA page; the system validates by comparing hashed input against stored hash.

- Maximum 3 OTP verification attempts allowed before requiring new OTP generation.

Session Management:

- Upon successful 2FA, the backend generates a JSON Web Token (JWT) containing:
  - user_id, role, email, issued_at, expires_at (15-minute expiration for access tokens).
  - Signed using HS256 (HMAC with SHA-256) with a secret key stored securely in environment variables (never hardcoded or committed to version control).
- Refresh Tokens (30-day expiration) are issued alongside access tokens, stored in the Sessions table with hashed values to enable revocation. Refresh tokens are HTTP-only cookies, preventing JavaScript access and mitigating XSS risks.
- Token Validation Middleware: All API endpoints (except login/2FA) require valid JWT in the Authorization: Bearer <token> header. Middleware verifies signature, checks expiration, and decodes user context before allowing request processing.
- Session Revocation: Users can explicitly log out, causing the backend to delete the corresponding session record from the Sessions table and Redis cache, invalidating the refresh token. Administrators can force-logout users by deleting their sessions.

4.3.6.2. Authorization and Access Control

Role-Based Access Control (RBAC) :

Authorization is enforced at two levels:

1. API Endpoint Level:
   - Each API endpoint specifies required roles in route decorators.
   - Authorization middleware extracts the role claim from the validated JWT and verifies it against endpoint requirements before executing the request handler

- Unauthorized access attempts return HTTP 403 Forbidden with audit log entry.

2. Data-Level Access Control:
   - Database queries include role-based filters ensuring users only access data permitted by their role.
   - Critical Constraint: Administrators (role=Admin) are explicitly prohibited from accessing the Diagnoses table.

Resource	Radiologist	Doctor	Admin
Upload X-ray	Write	Not allowed	Not allowed
Review AI Predictions	Read / Write	Read	Not allowed
Submit Diagnosis	Not allowed	Write	Not allowed
Generate Report	Not allowed	Write	Not allowed
View Patient Data	Read	Read	Not allowed
User Management	Not allowed	Not allowed	Read / Write
Access Logs	Not allowed	Not allowed	Read

Table 22 Data level access control

4.3.6.3. Data Encryption

Encryption in Transit:

- Client-to-Server: All communication between user browsers and the Load Balancer occurs over HTTPS with TLS 1.2+ (TLS 1.3 preferred for enhanced security and performance). SSL certificates issued by trusted Certificate

Authorities (e.g., Let's Encrypt, DigiCert) with automatic renewal via ACME protocol.

- Service-to-Service: Internal communication between Backend API Servers, Database, Redis, AI Inference Service, and Object Storage uses TLS 1.2+ with mutual TLS (mTLS) for critical paths (e.g., API to Database) to authenticate both client and server.
- Email Transmission: OTP codes and notifications sent via Email Service use TLS-encrypted SMTP (STARTTLS) to protect emails in transit.

Encryption at Rest:

1. Database Encryption:

- PostgreSQL configured with Transparent Data Encryption (TDE) using AES-256 encryption for tablespace storage, protecting data files, backups, and write-ahead logs (WAL).
- Field-Level Encryption for highly sensitive fields:
  - Users.password_hash: Hashed (Argon2id/bcrypt), not encrypted (one-way function).
  - Diagnoses.diagnosis_notes, Diagnoses.treatment_recommendations: Encrypted using AES-256-GCM
  - Audit_Logs.details: Encrypted (AES-256-GCM) to protect sensitive action metadata from unauthorized access.

2. Object Storage Encryption:

- X-ray images, annotated images, and PDF reports stored in S3/Azure Blob with server-side encryption (SSE) using AES-256.
- Pre-signed URLs for file downloads include expiration times (e.g., 15 minutes) and are generated only for authenticated users with appropriate roles, preventing unauthorized file access.

3. Backup Encryption:

- Daily and weekly backups encrypted with AES-256 before storage in cross-region S3 buckets.

- Backup encryption keys managed separately from production keys to prevent compromise of both systems simultaneously.

4.4. Verifying the Requirements in the Design

This section demonstrates how the proposed system design fulfills all functional and non-functional requirements specified in Chapter Three. A systematic verification ensures the design aligns with stakeholder needs, addresses identified gaps, and resolves any discrepancies discovered during the design phase. The verification employs traceability matrices, design walkthroughs, and requirement validation methods to provide confidence that the implemented system will meet its intended objectives.

4.4.1. Functional Requirements Verification

The following table traces each functional requirement to specific design components, demonstrating how the architecture satisfies user needs:

Requirement	Design Component	Verification Method
FR-08: Image Preprocessing	Image Preprocessor in AI Inference Subsystem	GPU-accelerated preprocessing resizes to 640×640
FR-09: AI-Based Disease Detection	YOLOx Inference Engine	YOLOx model classifies regions into Pneumonia, Tuberculosis, Lung Tumor, Normal; outputs class labels and confidence scores
FR-12: Inference Result Storage	Result Repository; Inference_Results table	Stores image_id, model_version, processing_time_sec, predictions JSONB, created_at; enables radiologist review

Table 23 AI Inference & Analysis functional Requirements Verification

Requirement	Design Component	Verfication Method
FR-13: Review System Predictions	Radiologist Review UI	Displays X-ray with overlayed bounding boxes, confidence scores, class labels; fetches data from Inference_Results via Case Repository.
FR-14: Edit Detection Annotation	Annotation Editor component; interactive bounding box tools	Radiologists resize, move, delete bounding boxes; add new regions; change disease classes
FR-15: Adjust Confidence Filters	Filter Engine; threshold slider (0-100%) on Review UI	Slider dynamically filters displayed annotations based on confidence threshold

Table 24 Radiologist Workflow functional Requirements Verification

Requirement	Design Component	Verification Method
FR-17: Final Diagnosis Submission	Diagnosis Controller; Diagnosis Form UI	Doctor selects primary_diagnosis, enters diagnosis_notes, urgency_level (Critical/Non-Critical), treatment_recommendations; encrypted and stored in Diagnoses table.
FR-21: Download Report	Report Generator	Generates PDF with original image, annotations, findings,

as PDF		diagnosis
FR-22: View Past Reports	Case History Service; Patient Search UI	Doctors query historical cases by patient ID, diagnosis type, date range; results display all past diagnoses and reports.

Table 25 Doctor Workflow functional Requirements Verification

Requirement	Design Component	Verfication Method
FR-26: User Management	Admin Controller; User Management UI	Admins add users (generates temporary password sent via email), deactivate reset credentials, assign roles; actions logged in Audit_Logs.
FR-27: Access Log Viewing	Audit Log Viewer; Access Logs UI with filtering	Displays immutable Audit_Logs filtered by action type, case ID, user ID, date range; supports pagination for large datasets.

Table 26 Administrative Requirements functional Requirements Verification

4.4.2. Non-Functional Requirements Verification

The following table verifies how the design addresses each non-functional requirement category:

Requirement	Design Solution	Verification Method
GPU-accelerated AI	GPU-accelerated AI	Inference workflow

Inference Service; YOLOx lightweight architecture; asynchronous processing via message queue	Inference Service; YOLOx lightweight architecture; asynchronous processing via message queue	(upload → validation → preprocessing → YOLOx forward pass → storage) completes within 12s on NVIDIA Tesla GPU; benchmarked during testing phase.
NFR-2: Interface Interaction Time (<300ms)	Redis caching for case data, report metadata; optimized React rendering; CDN for static assets	UI interactions (toggle overlays, filter annotations) render <300ms via cached data retrieval and client-side rendering optimizations; measured using browser DevTools Performance profiler.
NFR-3: Report Generation Time (≤10s cached, ≤18s non-cached)	Report caching in Redis; pre-compiled PDF templates; PDF Engine using ReportLab	Cached reports retrieved from Redis in ≤10s; non-cached reports generated via compiled templates with embedded images in ≤18s; measured with Prometheus metrics.
NFR-4: Concurrent Session Support (200+ users)	Horizontal scaling of API servers (3-5 instances); stateless design with Redis session store	Load testing (Apache JMeter, Locust) simulates 200+ concurrent users; system maintains <300ms response times without failure; Kubernetes auto-scales API pods based on CPU utilization.

Table 27 Performance Requirements

Requirement	Design Solution	Verification Method
NFR-5: Uptime (≥95%)	Multi-AZ deployment; automatic health checks; failover mechanisms	Uptime monitored via Prometheus; PostgreSQL primary-replica failover tested; API server restarts on failure (Kubernetes livenessProbe)
NFR-6: Maintenance Time (3 hours/month for model updates)	AI Inference Service independent scaling; model versioning; blue-green deployment	Model updates deployed to staging AI nodes first; traffic gradually shifted from old to new model version
NFR-7: Automatic Retry for Inference	Message queue (Redis/RabbitMQ) with retry logic (max 1 retry); failed jobs queue for admin review	Transient inference failures automatically retry once; persistent failures enqueued to admin dashboard showing failed jobs with error logs; tested via chaos engineering.
NFR-8: Automatic Recovery	Kubernetes automatic pod restarts; health probes	Unhealthy API server containers automatically terminated and restarted; tested by manually killing pods and verifying self-healing within 30 seconds.
NFR-9: Scheduled Backups	Automated cron jobs for daily incremental and weekly full PostgreSQL backups	Backup Service executes pg_dump for full backups (Sunday 2 AM) and incremental backups (daily 2 AM); backup files verified via test restores; monitored via CloudWatch/Prometheus.

Table 28 Reliability & Availability

Chapter Five: System Implementation

5.1. Reviewing the Design Solution

This section revisits the design solution presented in Chapter Four, evaluating its alignment with the project's objectives and ensuring design decisions remain valid as the project transitions into the implementation phase.

5.1.1. Design Alignment with Project Objectives

The proposed AI-powered lung disease detection system was designed to address three primary objectives: enhancing diagnostic accuracy, improving accessibility, and reducing diagnostic delays in Ethiopia's healthcare system. Upon review, the layered architecture combining microservices principles successfully supports these goals through:

1. Real-Time AI Analysis: The YOLOx-based inference service, decoupled from the main backend, enables parallel processing of multiple X-ray images while maintaining the ≤12-second performance constraint (NFR-1). The selection of YOLOx-M over larger variants (YOLOx-X) represents a validated trade-off between accuracy (mAP 46.9%) and speed, ensuring clinical utility without sacrificing diagnostic reliability.
2. Scalable Cloud Architecture: The deployment design utilizing containerized services, load balancing, and horizontal scaling supports the objective of nationwide accessibility. The architecture accommodates 200+ concurrent users (NFR-4) across distributed hospitals, with auto-scaling capabilities ensuring consistent performance during peak usage periods.
3. Clinical Workflow Integration: The role-based user interface design—tailored for Lab Technicians, Radiologists, Doctors, and Administrators—streamlines diagnostic workflows from image upload through AI-assisted review to final diagnosis submission. The explainable AI integration via Grad-CAM heatmaps addresses the literature-identified need for transparency, building clinician trust and facilitating adoption.

5.1.2. Validation of Key Design Decisions

Several critical design decisions made during Chapter Four were re-evaluated for implementation feasibility:

1. PostgreSQL for Primary Data Storage

- Rationale: ACID compliance essential for medical data integrity; robust support for complex queries (patient history searches, case filtering); JSONB type accommodates variable-structure AI predictions.
- Implementation Validation: Confirmed appropriate. PostgreSQL's Row-Level Security (RLS) provides database-level enforcement of admin diagnosis access restrictions (NFR-11), complementing application-layer RBAC. Indexing strategies (B-tree on foreign keys, GIN on JSONB) optimize query performance validated through benchmarking.

2. Separation of AI Inference as Independent Microservice

- Rationale: GPU resource isolation; independent scaling; model updates without API downtime; technology flexibility (Python/PyTorch for AI vs. FastAPI for backend).
- Implementation Validation: Confirmed essential. This separation enables blue-green deployment for model updates, maintaining ≥95% uptime (NFR-5). Asynchronous communication via message queues (Redis) prevents blocking user workflows during inference.

3. Two-Factor Authentication via Email OTP

- Rationale: Enhanced security for sensitive medical data access; low implementation cost; no dependency on SMS infrastructure in Ethiopia.
- Implementation Validation: Retained with modifications. Extended OTP validity to 5 minutes (from original 2 minutes) to accommodate email delivery latencies in low-bandwidth regions. Added "Resend OTP" functionality for improved user experience.

4. React.js + Next.js for Frontend

- Rationale: Component-based architecture facilitates reusable UI elements; server-side rendering improves initial load performance; TypeScript support enhances code maintainability.
- Implementation Validation: Confirmed optimal. Next.js App Router provides built-in routing, middleware for authentication checks, and optimized image loading—critical for handling medical images. TypeScript catches type errors during development, reducing runtime bugs in production.

5.2. Deciding on the Development Tools

This section details the selection of development tools, programming languages, frameworks, and libraries used to build the system. Tool selection prioritized open-source availability, community support, compatibility with existing infrastructure, and team expertise.

5.2.1. Programming Languages and Core Frameworks

1. Backend Development: Python 3.11 with FastAPI
    - Justification: Python's rich ecosystem for AI/ML (PyTorch, NumPy, scikit-learn) enables seamless integration between backend logic and AI inference. FastAPI's lightweight, unopinionated design allows custom architecture without framework constraints. Version 3.11 provides performance improvements (10-25% faster than 3.9) and enhanced type hinting.
    - Installation: pip install "fastapi[standard]"
2. AI Inference: Python 3.11 with PyTorch 2.1 and YOLOX
    - Justification: PyTorch's dynamic computation graphs facilitate model debugging and experimentation. YOLOX (You Only Look Once eXtended) offers state-of-the-art real-time object detection with customizable backbones (CSPDarknet) and neck architectures (PANet) for multi-scale feature fusion.
    - Installation: pip install torch==2.1.0 torchvision==0.16.0 --index-url https://download.pytorch.org/whl/cu118 (CUDA 11.8 for GPU acceleration)
    - YOLOX Setup: Cloned official repository git clone https://github.com/Megvii-BaseDetection/YOLOX.git and installed dependencies pip install -r requirements.txt

3. Frontend Development: TypeScript 5.2 with Next.js 14 and React 18
    - Justification: Next.js 14 App Router provides file-based routing, server components for reduced client-side JavaScript, and built-in API routes. TypeScript enforces type safety, reducing runtime errors in complex medical UI interactions. React 18's concurrent rendering improves UI responsiveness during AI prediction visuaization.
    - Installation: npx create-next-app@latest ui-client --typescript --tailwind --app

References

[1] Ahmed, A. I. Siam, A. E. M. Atwa, M. A. Atwa, E. Md. Abdelrahim, and E.-S. Atlam, "An Explainable YOLO-Based Deep Learning Framework for Pneumonia Detection from Chest X-Ray Images," Algorithms, vol. 18, no. 11, p. 703, Nov. 2025, doi: 10.3390/a18110703.

[2] B. Zhao, L. Chang, and Z. Liu, "Fast-YOLO Network Model for X-Ray Image Detection of Pneumonia," Electronics, vol. 14, no. 5, p. 903, Feb. 2025, doi: 10.3390/electronics14050903.

[3] J. Ma, Y. Song, X. Tian, Y. Hua, R. Zhang, and J. Wu, "Survey on deep learning for pulmonary medical imaging," Front. Med., vol. 14, no. 4, pp. 450–469, Dec. 2019, doi: 10.1007/s11684-019-0726-4.

[4] R. Bista et al., "Advancing Tuberculosis Detection in Chest X-rays: A YOLOv7-Based Approach," Information, vol. 14, no. 12, p. 655, Dec. 2023, doi: 10.3390/info14120655.

[5] Q. Zeng, T. Hu, Z. Chen, J. Zheng, J. Li, and Y. Pan, "YOLO-ED: An efficient lung cancer detection model based on improved YOLOv8," PLoS One, vol. 20, no. 9, p. e0330732, Sept. 2025, doi: 10.1371/journal.pone.0330732.

[6] A. M. Nigatu et al., "Effect of teleradiology on patient waiting time and service satisfaction in public hospitals, Northwest Ethiopia: a quasi-experimental study," BMC Health Serv Res, vol. 25, no. 1, Apr. 2025, doi: 10.1186/s12913-025-12545-8.

[7] Devex Editor, "Opinion: How AI can rejuvenate imaging equipment to ramp up TB screening," Devex, Mar. 19, 2025. https://www.devex.com/news/sponsored/opinion-how-ai-can-rejuvenate-imaging-equipment-to-ramp-up-tb-screening-109581 (accessed Dec. 09, 2025).

[8] "ASCENT Ethiopia, Research Results Dissemination and Closeout Event – ASCENT," ASCENT, 2020. https://www.digitaladherence.org/ascent-ethiopia-dissemination-meeting-and-closeout-2023/ (accessed Dec. 11, 2025).

[9] K. Tegenu, G. Geleto, D. Tilahun, E. Bayana, and B. Bereke, "Severe pneumonia: Treatment outcome and its determinant factors among under-five patients, Jimma,

Ethiopia," SAGE Open Medicine, vol. 10, no. 2, p. 205031212210784, Jan. 2022, doi: https://doi.org/10.1177/20503121221078445.

[10] A. A. Buser, "The Training and Practice of Radiology in Ethiopia: Challenges and prospect," Ethiopian Journal of Health Sciences, vol. 32, no. Spec Iss 1, p. 1, Oct. 2022, doi: https://doi.org/10.4314/ejhs.v32i1.1S.

[11] A. Brady, R. Ó. Laoide, P. McCarthy, and R. McDermott, "Discrepancy and Error in Radiology: Concepts, Causes and Consequences," The Ulster Medical Journal, vol. 81, no. 1, p. 3, 2012, Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC3609674/

[12] Nebiyu Mekonnen Derseh et al., "Incidence rate of mortality and its predictors among tuberculosis and human immunodeficiency virus coinfected patients on antiretroviral therapy in Ethiopia: systematic review and meta-analysis," Frontiers in Medicine, vol. 11, Apr. 2024, doi: https://doi.org/10.3389/fmed.2024.1333525.